

# AI (CM12001)

Semester 2, Week 2: Markov Decision Processes

Emelie Barman

13 February 2025

Thank you to Dr Ben Ralph for parts of the slides

# Sequential decision making under uncertainty



# Pacman

- How do you make decisions in a game like Pacman?
- The game can be modelled as a state space, just like you saw in the first semester.
- However, the movement of the enemies means that we don't know what state will be reached by each action taken.
- Therefore, we need to use probability.



# Tetris

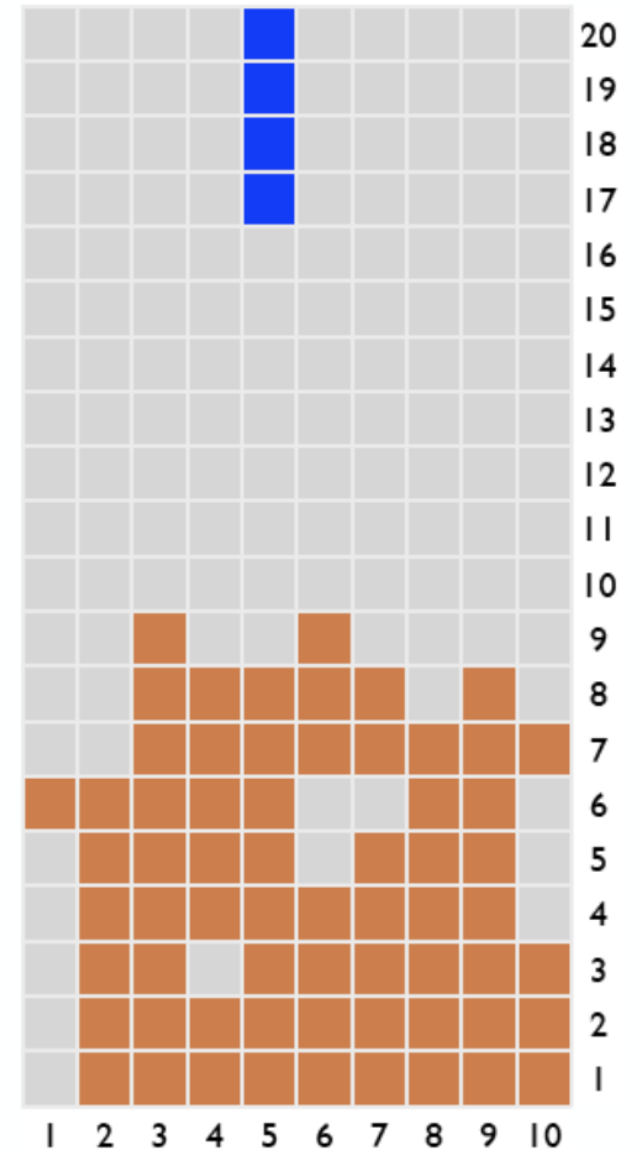
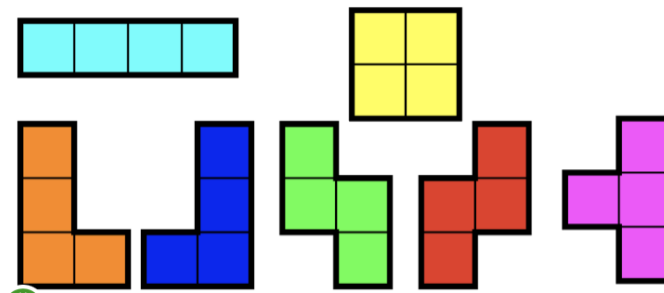
- One out of seven pieces is appearing at the top.
- The player decides where to place it.
- If an entire row is filled, it disappears.
- The more rows that are cleared simultaneously, the higher is the reward;

1 row 1p

2 rows 3p

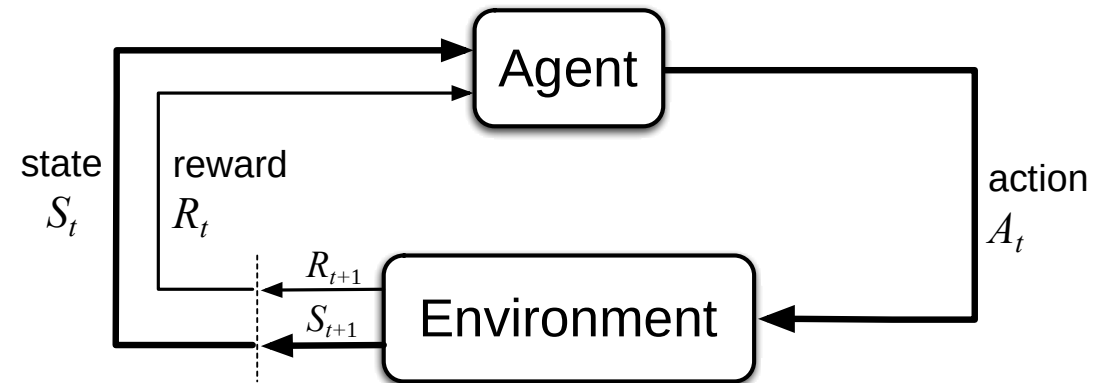
3 rows 5p

4 rows 8p



# States, actions and rewards

- We can model probabilistic systems in a similar way to the usual state spaces we saw last semester.
- The key difference is that actions no longer have guaranteed outcomes.
- It is also usual to have some notion of reward that is awarded every time the consequence of an action is discovered.
- The probability of ending up in a specific new state is called the **transition probability**.

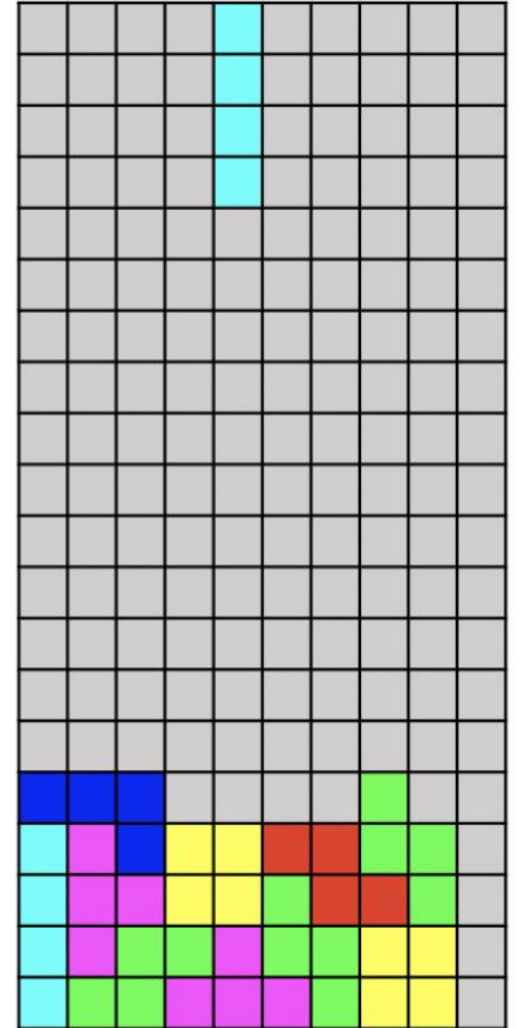
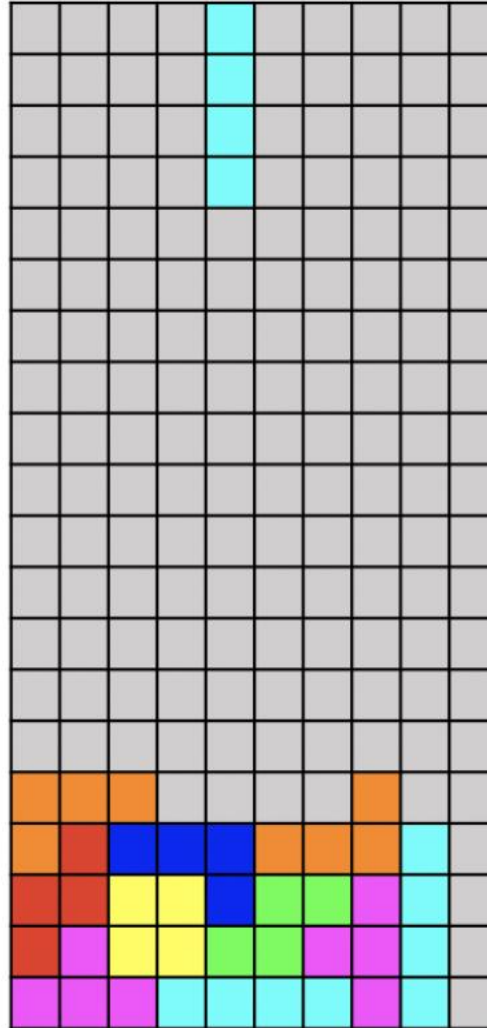


# The Markov Property

- In some situations, it might be reasonable to assume that every previous state and action influences the transition probability.
- $p(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0)$
- However, this would make the probabilities a nightmare to work with in practice.
- Therefore, we isolate a special property of probabilistic systems which have the **Markov Property**.
- $p(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(s_{t+1} | s_t, a_t)$ .
- Systems with the Markov Property are “memoryless” – knowing how a particular state was arrived adds no information to what will happen next.

# Tetris

- The path taken to reach current state does not affect what further actions we should take.
- In terms of choosing future actions – the states to the right can be seen as equivalent.

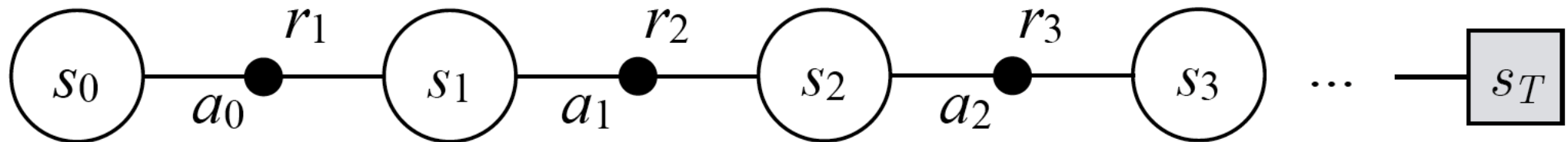


# Markov Decision Processes

Using the Markov Property, we can define a **Markov Decision Process** (MDP)

A Markov Decision process consists of:

- A set of states:  $\{s_0, s_1, \dots\}$
- Actions from each state:  $\{a_0, a_1, \dots\}$
- Transition probability  $p(s_{t+1} | s_t, a_t)$
- A Reward for each transition  $r(s_{t+1}, s_t, a_t)$





# Transition probabilities

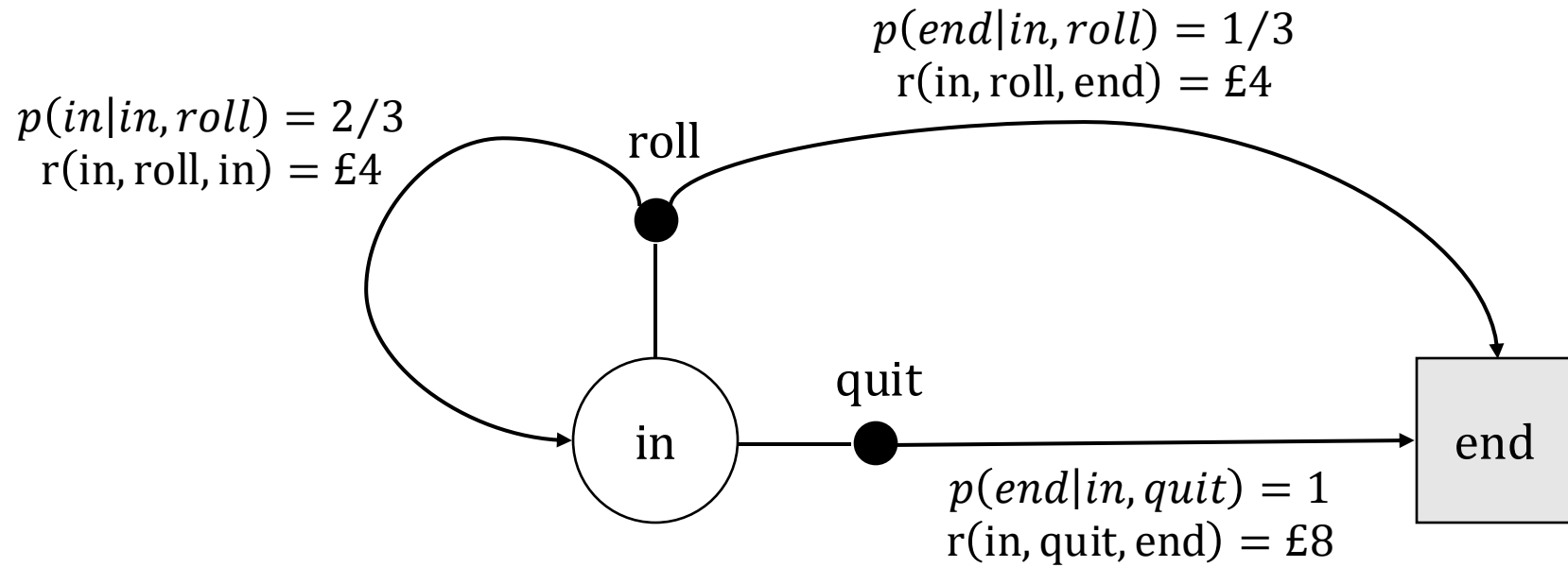
- The transition probabilities  $p(s_{t+1} | s_t, a_t)$  specify the probability of arriving in state  $s_{t+1}$  from state  $s_t$  given action  $a_t$ .
- For a given state and action the transition probabilities should sum to 1.
- We can actually think of the search problems we saw last semester as a special case of MDPs, where  $p(s_{t+1} | s_t, a_t) = 1$  for exactly one new state and 0 for every other new state.

# A very simple game

- You are playing a game with a single die.
- You need to decide whether to roll the die or to quit.
  - If you quit, you get £8.
  - If you roll the die, and it is 1 or 2, you get £4 and reach the end of the game.
  - If it is any other number, then you get £4 and you stay in the game, and can choose what to do next.
- We can model this game using an MDP.



# An MDP for this game



# Maximizing the long-term reward

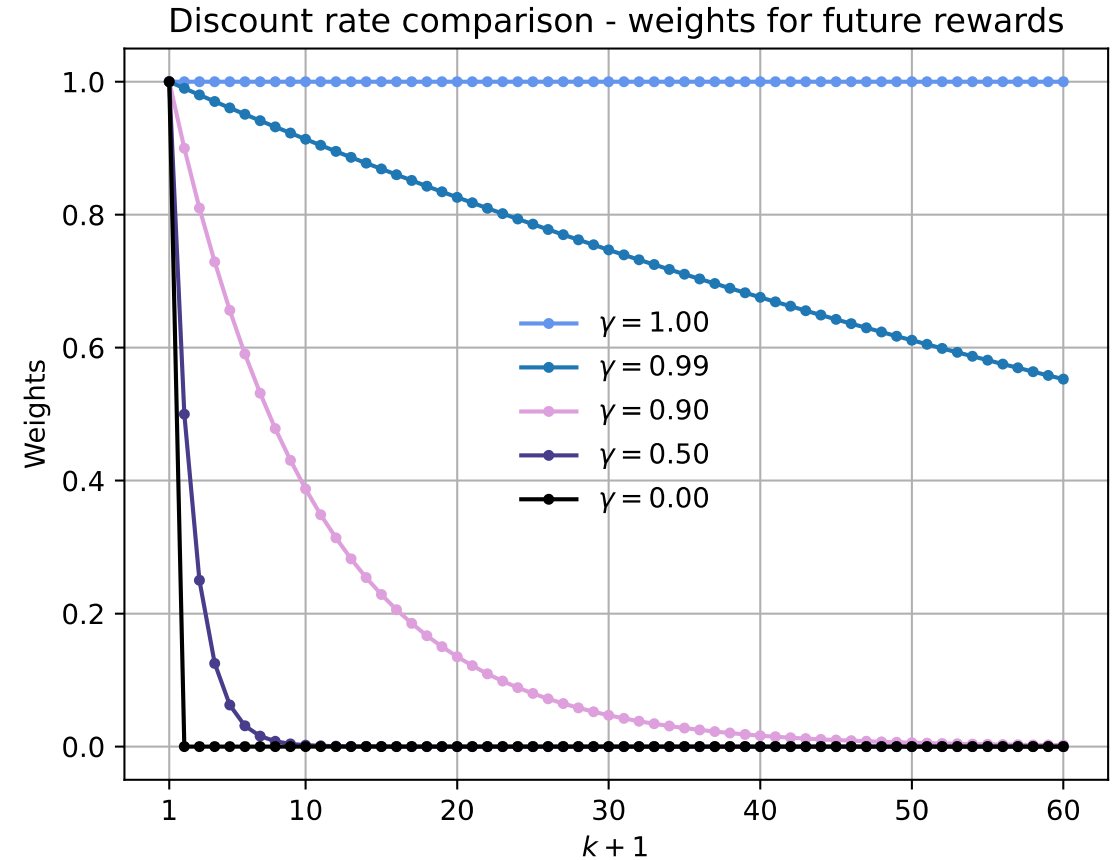
- How do we decide on a strategy for choosing actions in an MDP?
- Before we can do so, we need a way of evaluating sequences of states in an MDP.
- The **discounted return** with discount rate  $\gamma$  of a sequence of states is defined as the following:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$
$$0 \leq \gamma \leq 1$$

- The weights – the importance of future rewards.

# The discount rate

- A discount rate of  $\gamma = 1$  implies that a future reward has the same value as a present reward.
- A discount rate of  $\gamma = 0$  implies that a future reward has no value.
- The discount rate specifies how much we take future rewards into account.
- A lower discount rate motivates the decision maker to favour taking actions early, rather than postponing them.



# Aside: Social Discount Rates and Climate Policy

- Discount rates are not just a concept in Artificial Intelligence.
- The **Social Discount Rate (SDR)** is a concept used by economists to put a present value on costs and benefits that will fall on society in the future.
- SDRs have a big role in debates about climate policy among economists.
- Those who argue that we don't need to take drastic action on climate change often implicitly or explicitly base their arguments on a high SDR.
- Those who argue that we do need to take immediate action, even at a high cost to present society, will implicitly or explicitly assume a low SDR.



We have built a robot which is supposed to collect empty cans for recycling from around an office.

The robot decides its actions based only on the level of its battery, **high** or **low**. These are the **states**.







**In either state, it can do one of three actions:**



### **Search**

1. Roam around and search for empty cans that have been discarded on the ground.



### **Remain**

2. Remain stationary and wait for people to bring it cans.



### **Return**

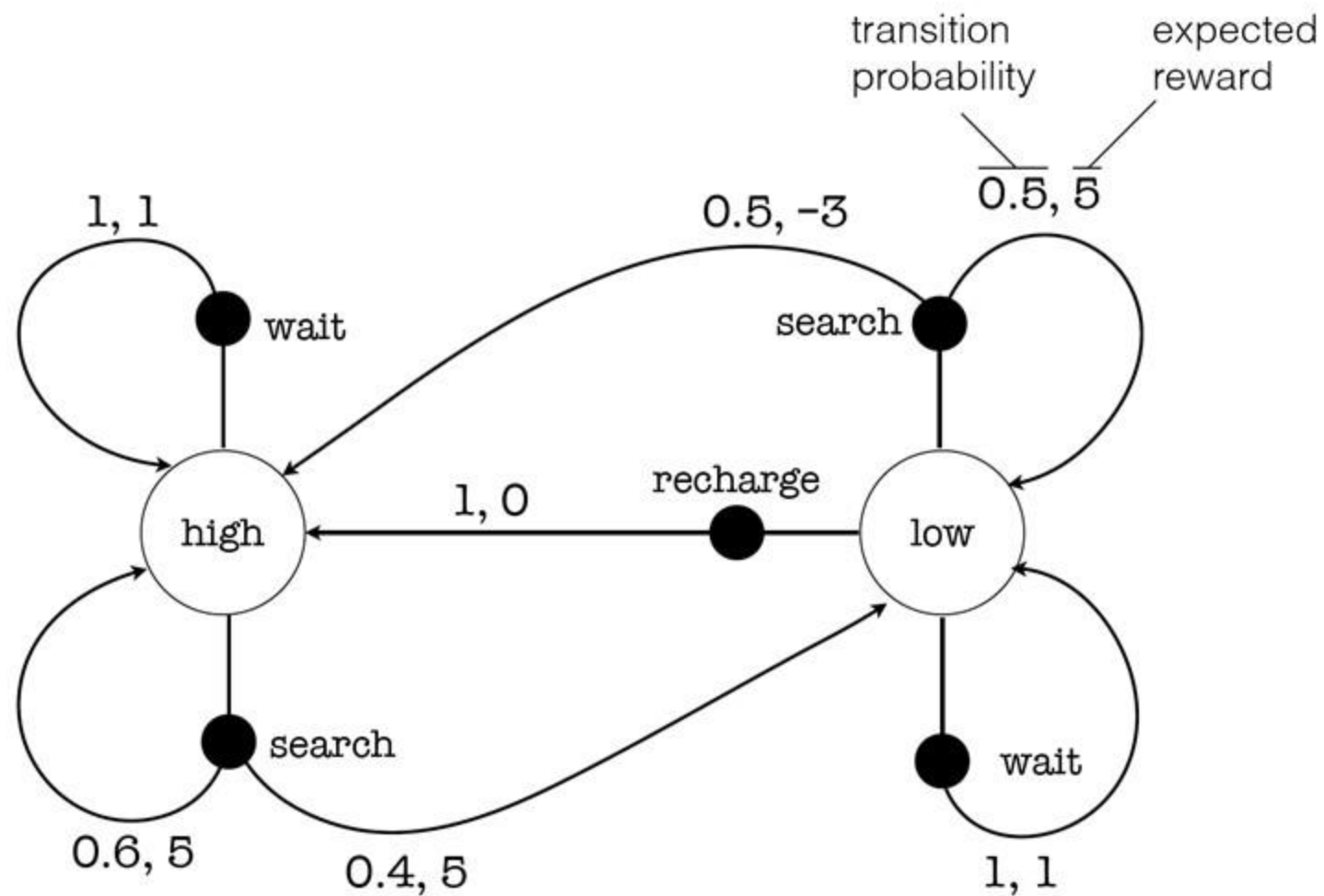
3. Return to its base and recharge its battery – this action is obviously only needed if the robot is in the low battery state.



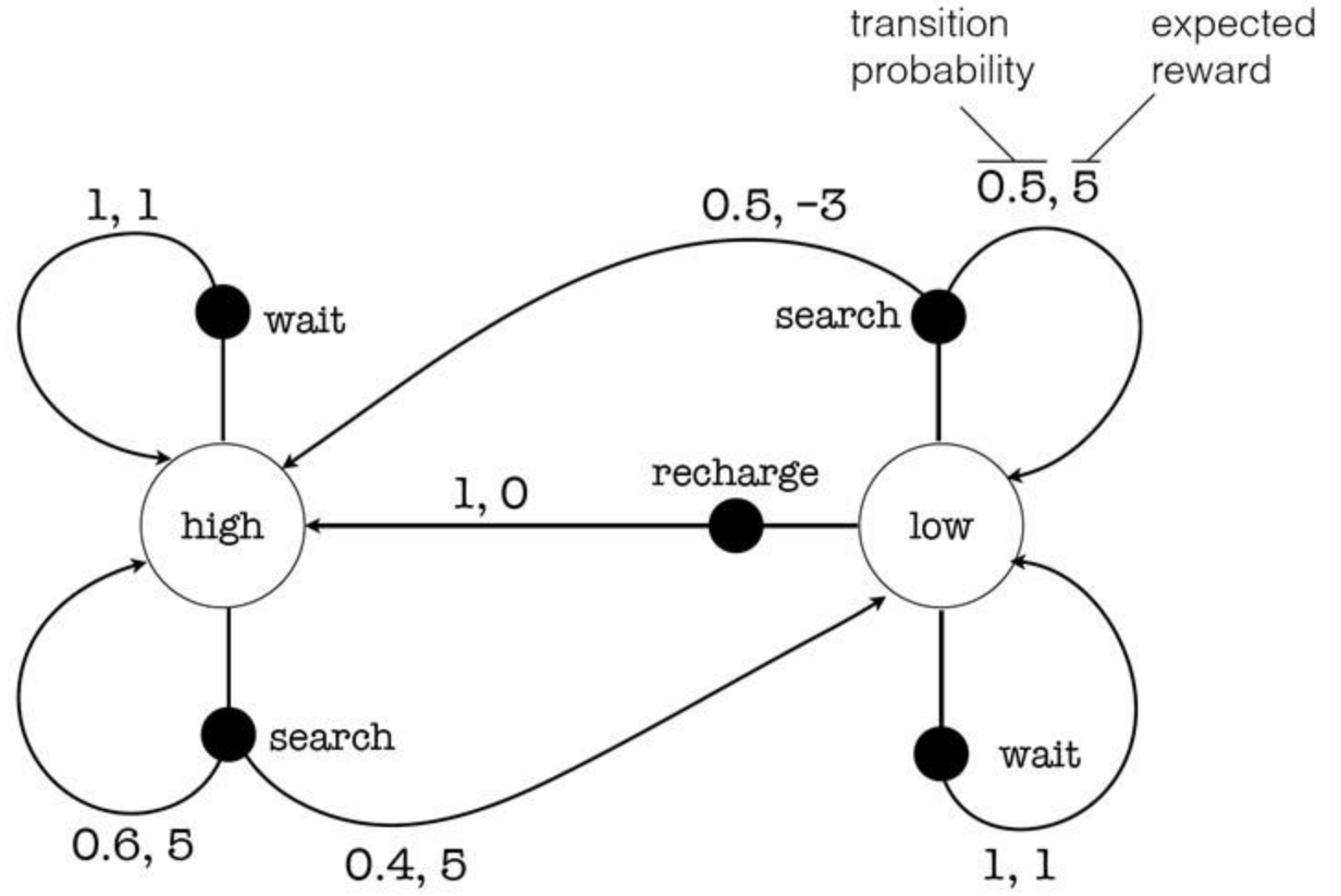
To model this as a Markov decision process we need the following:

- A set of states  $S$ 
  - $S = \{\text{low}, \text{high}\}$
- A set of actions available from each state  $A(s), s \in S$ 
  - $A(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$
  - $A(\text{high}) = \{\text{search}, \text{wait}\}$
- Finally for each state, action, and subsequent state, we need a transition model  $p(s'|s, a)$  and a reward function  $r(s, a, s')$  – the exact choice of these will determine the best actions for the robot.

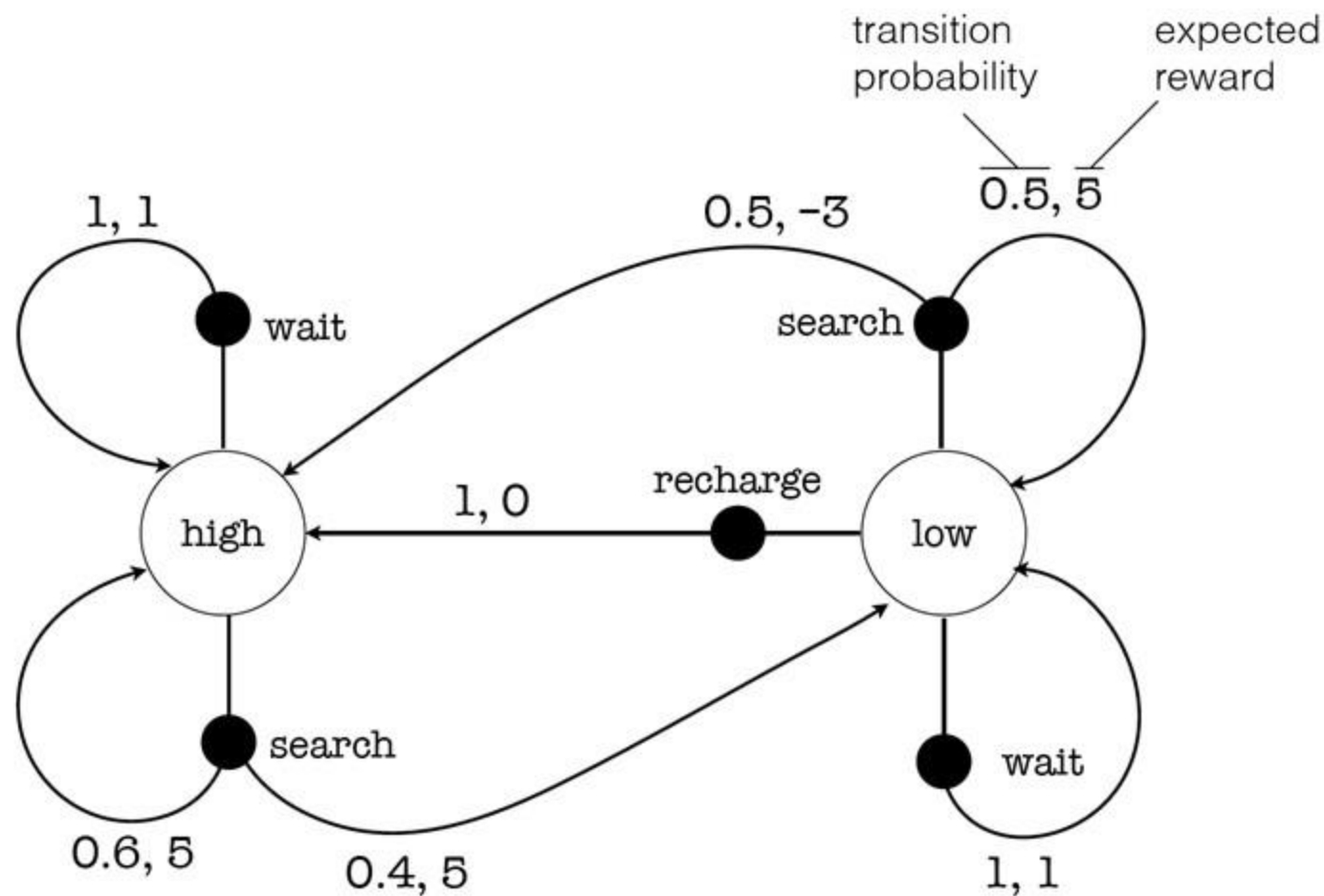
We can summarise the MDP, and define the final values, using a *transition graph*.



Large empty circles represent states, and small filled circles represent the actions available from each state.

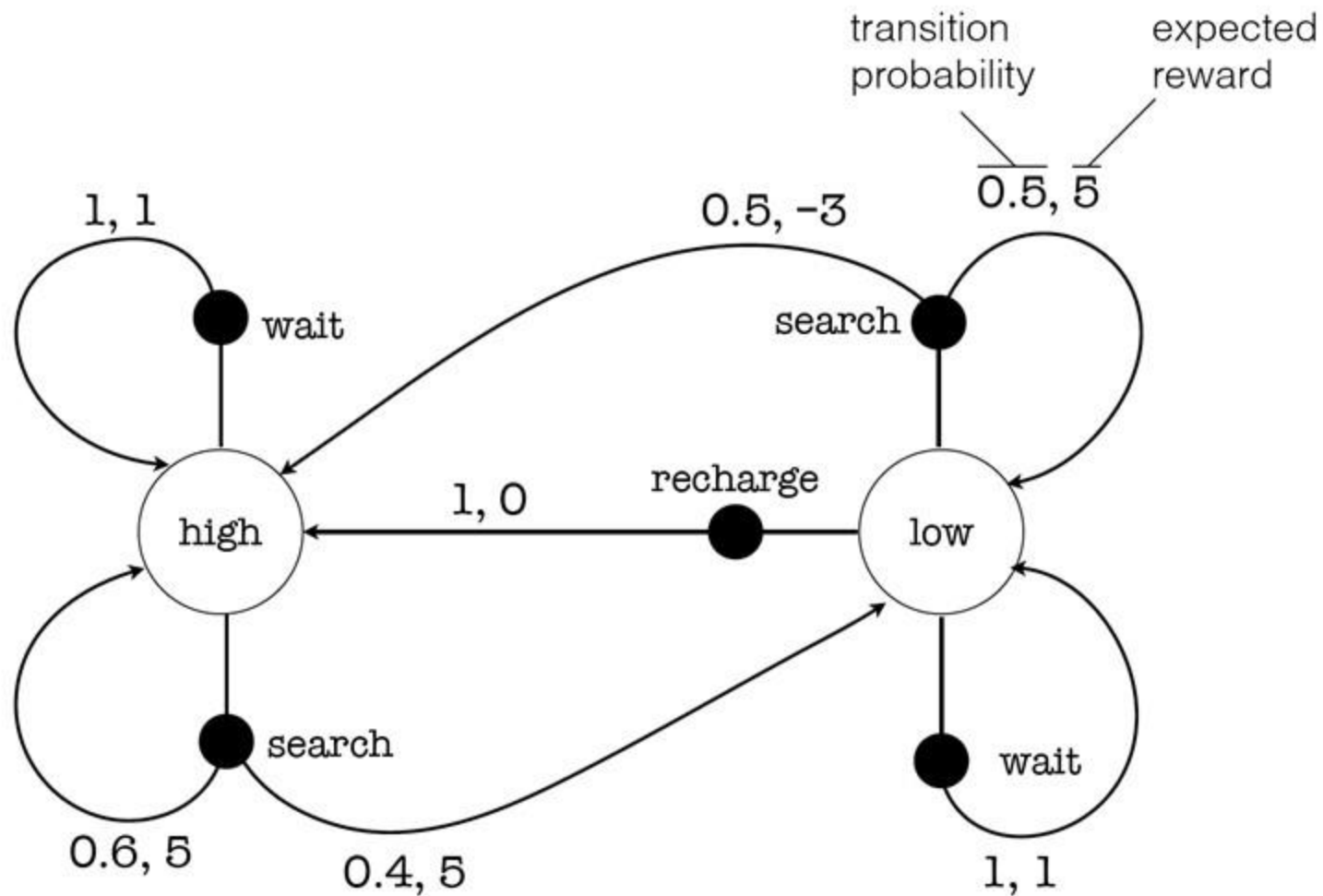


If the robot takes an action from either state then it will follow the direction of the relevant arrow, which can possibly split to indicate uncertainty. Notice that if the action arrow splits, the transition probabilities for each action must **sum to one**. If the action arrow does not split, indicating certainty in the action taken, then the transition probability is 1.



Each branch is labelled with a transition probability and an expected reward value, which are

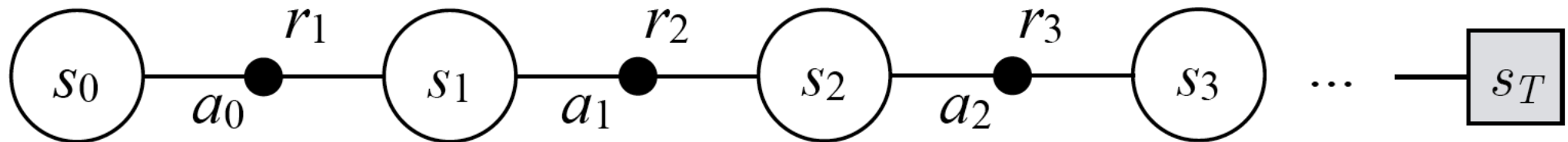
$$p(s'|s, a) \text{ and } r(s, a, s')$$

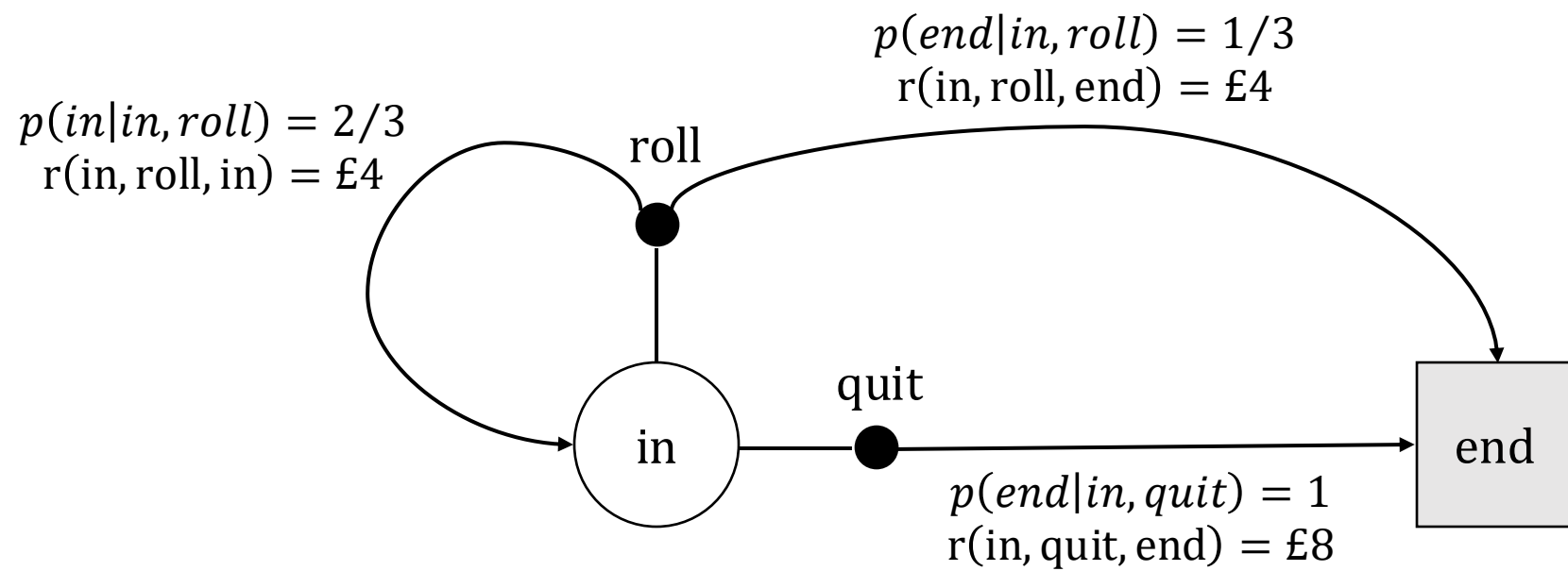


# Recap - Markov Decision Processes

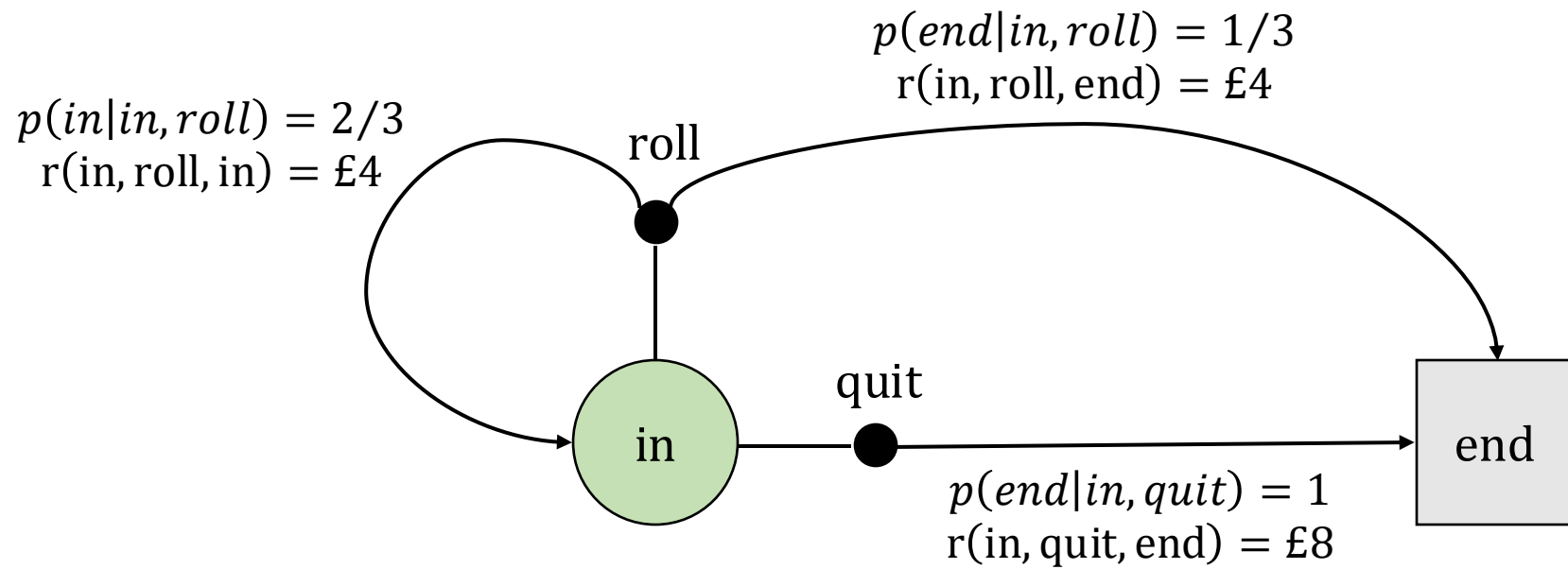
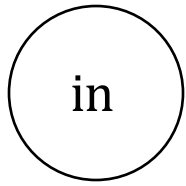
A Markov Decision process consists of:

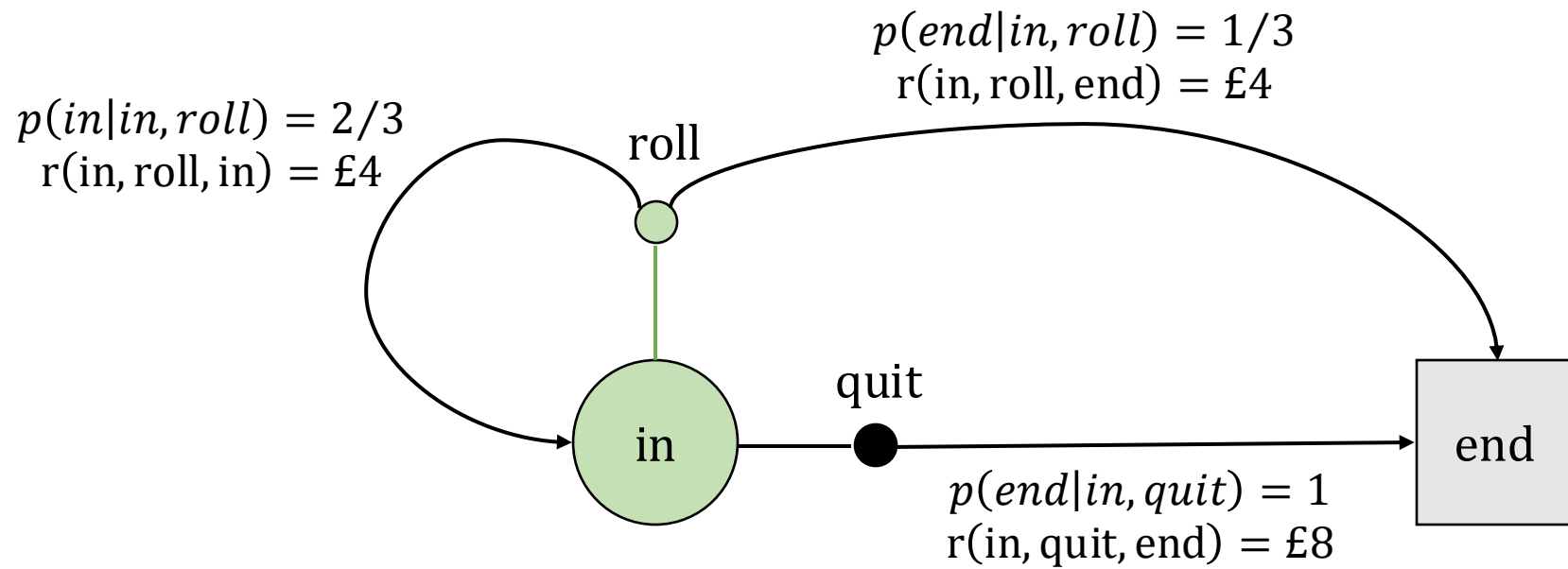
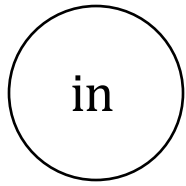
- A set of states:  $\{s_0, s_1, \dots\}$
- Actions from each state:  $\{a_0, a_1, \dots\}$
- Transition probability  $p(s_{t+1} | s_t, a_t)$
- A Reward for each transition  $r(s_{t+1}, s_t, a_t)$

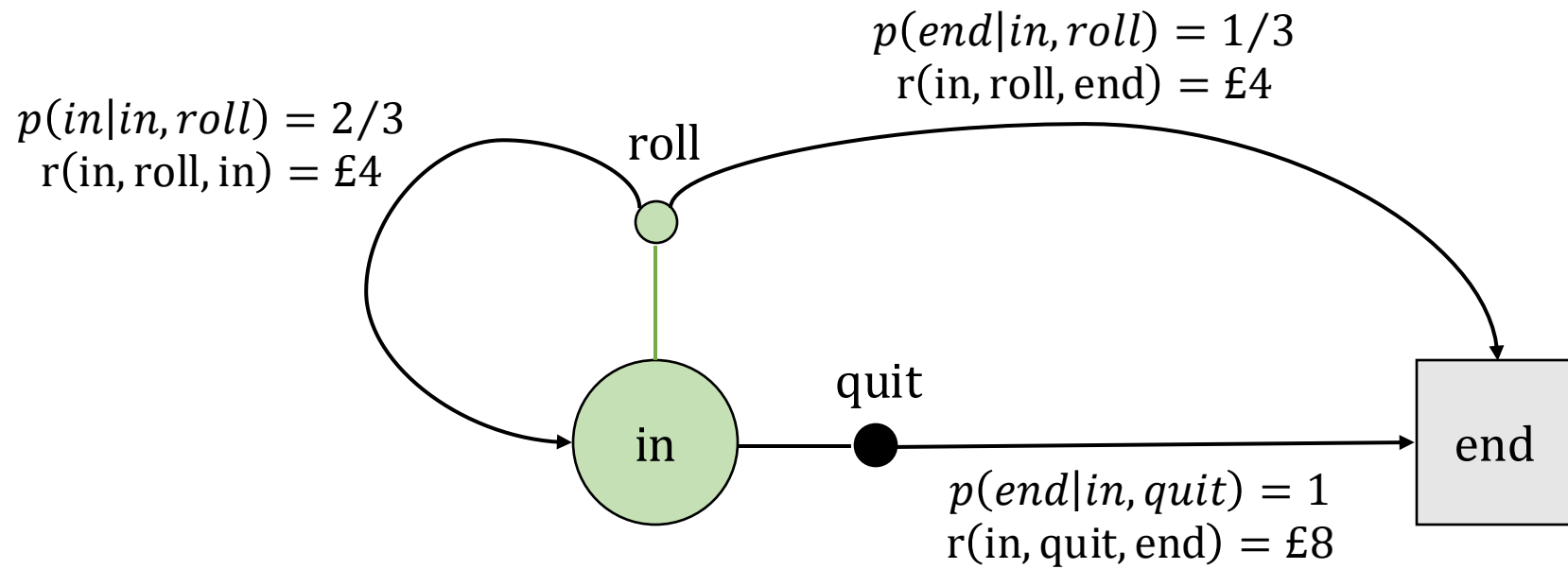
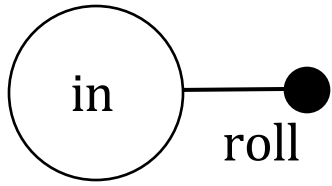


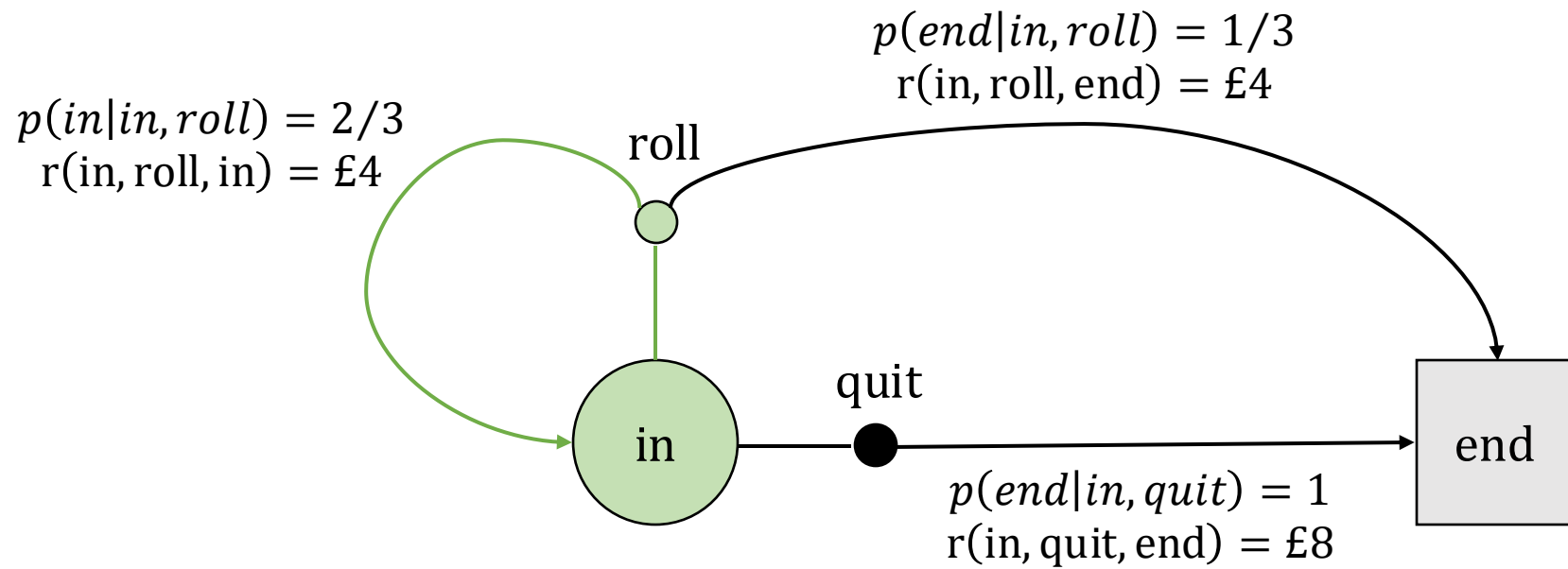
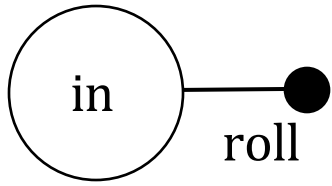


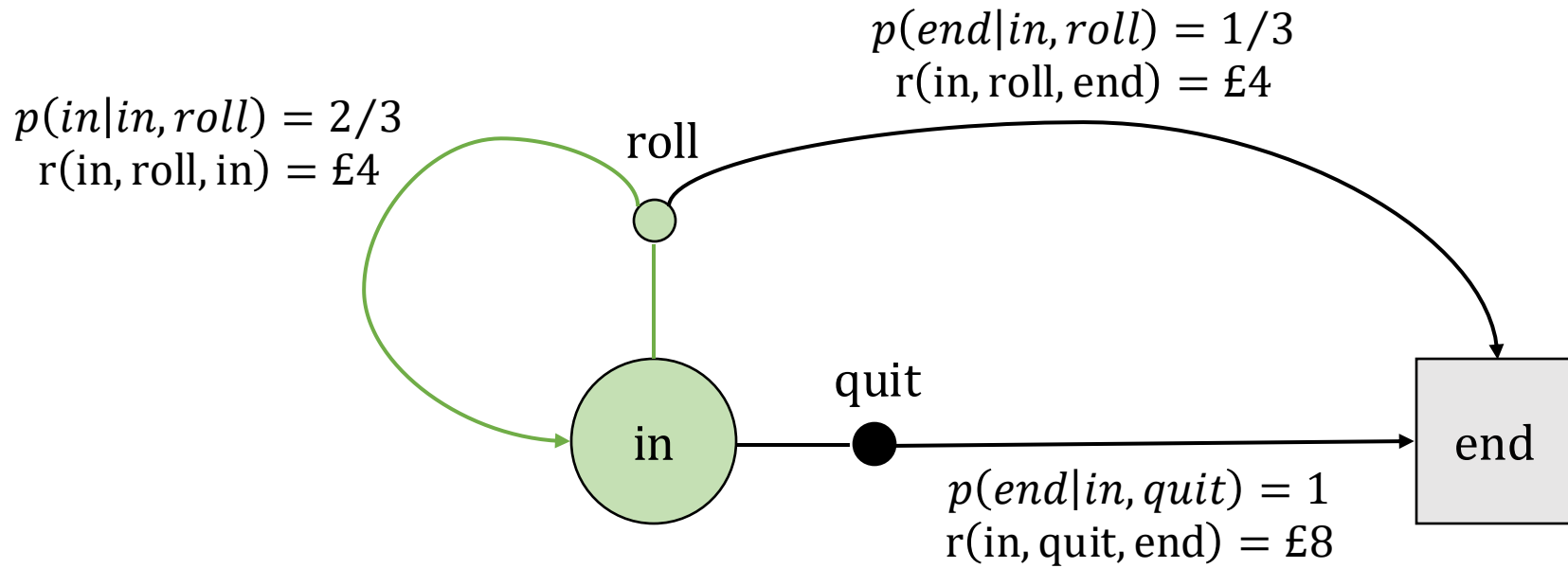
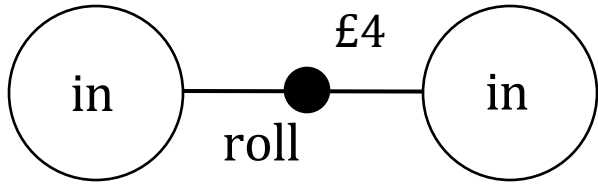


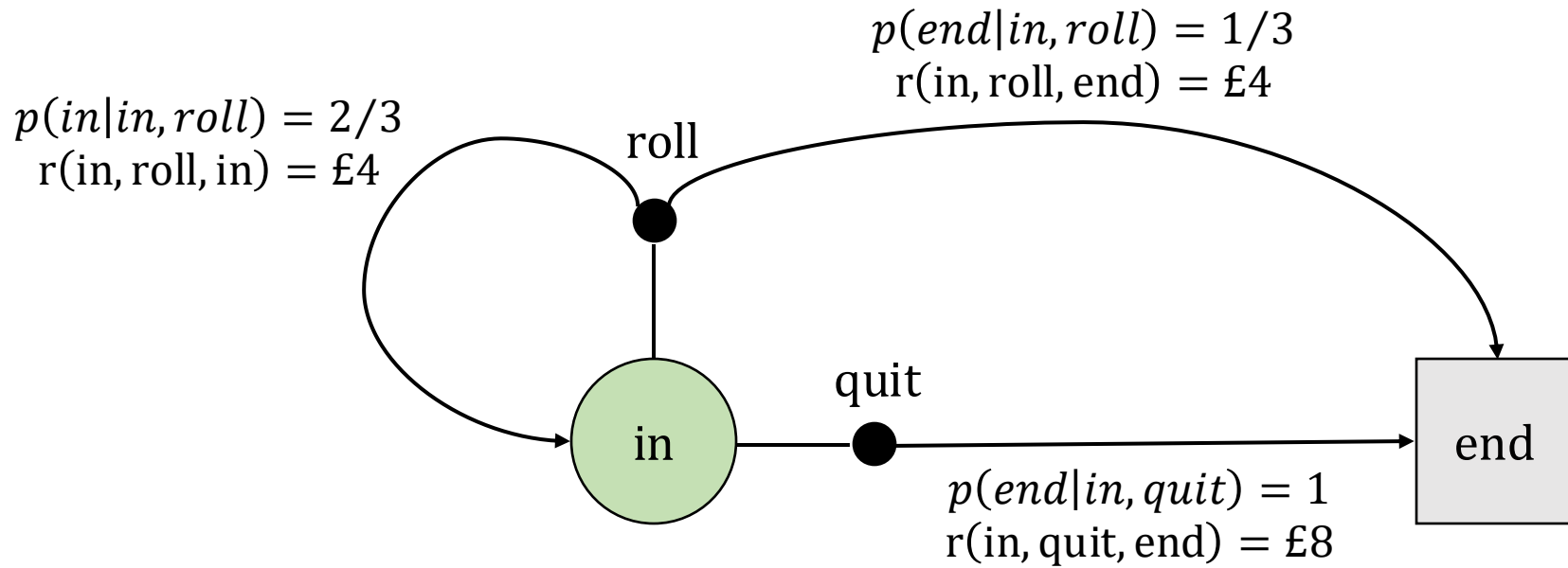
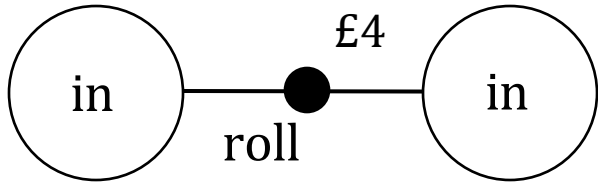


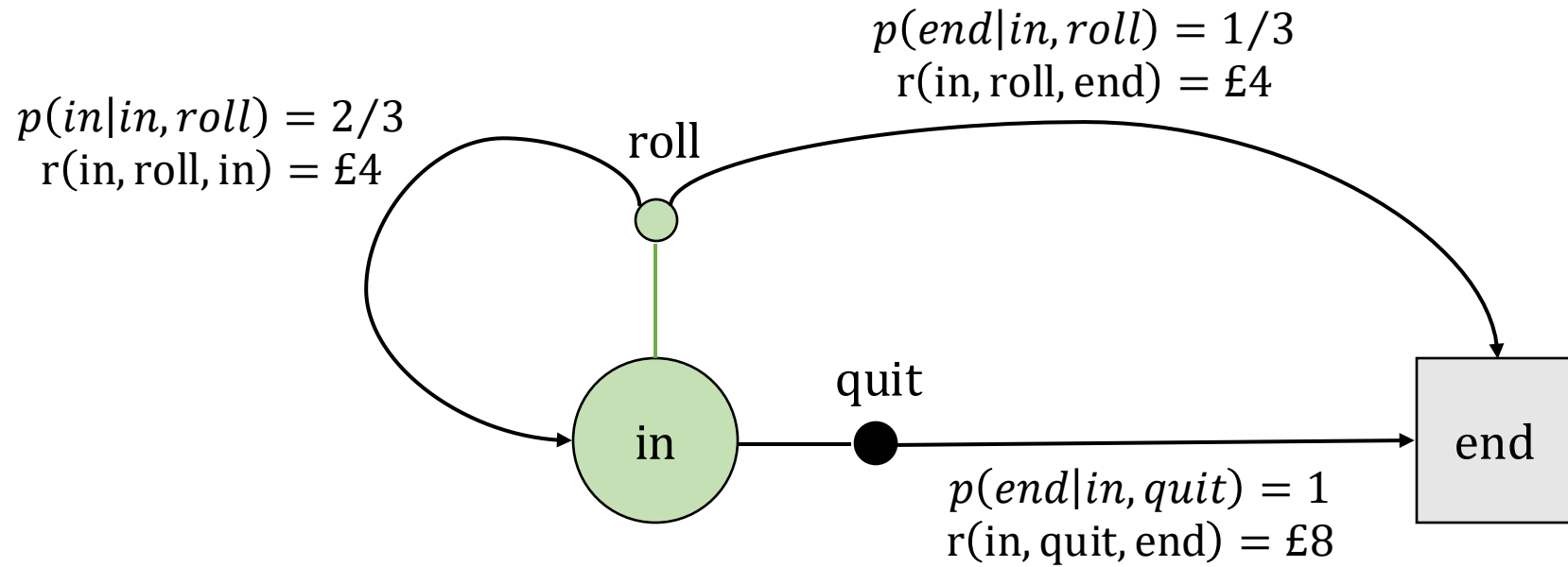
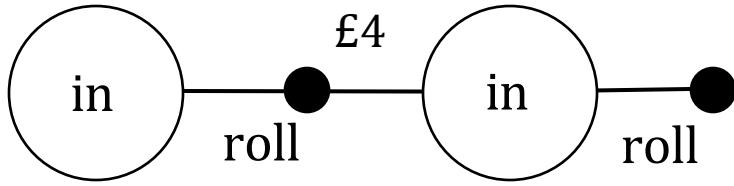


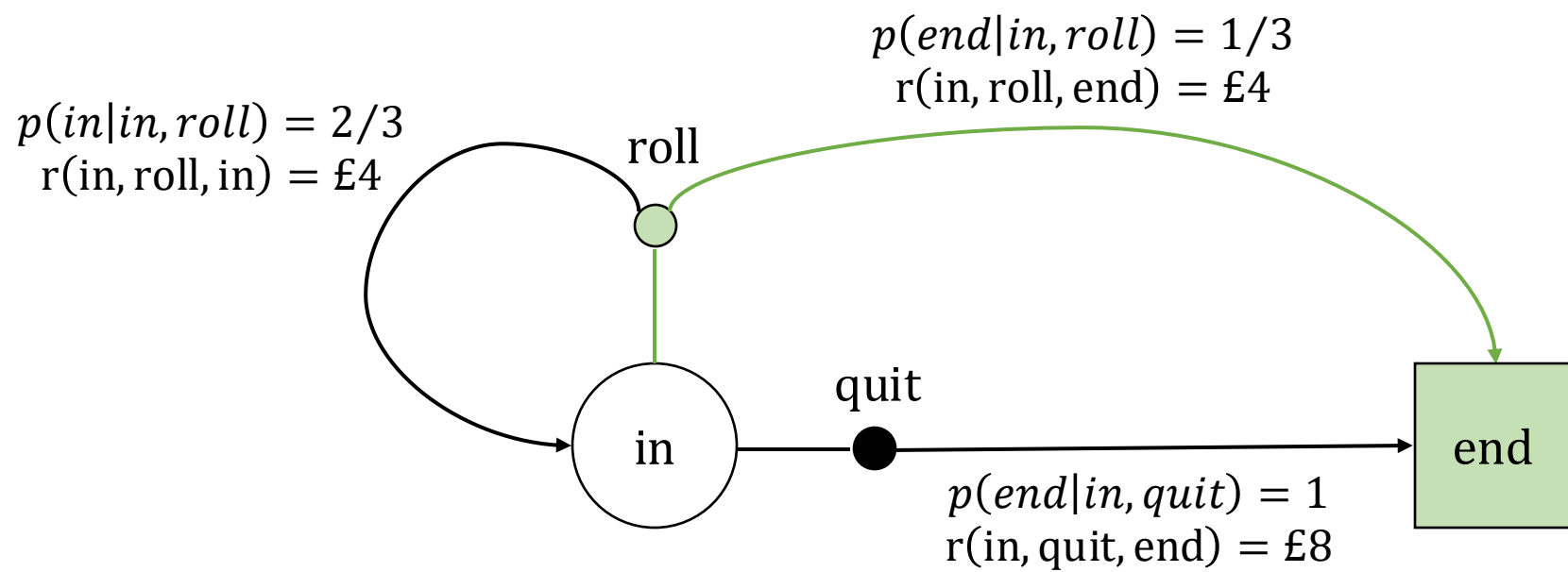
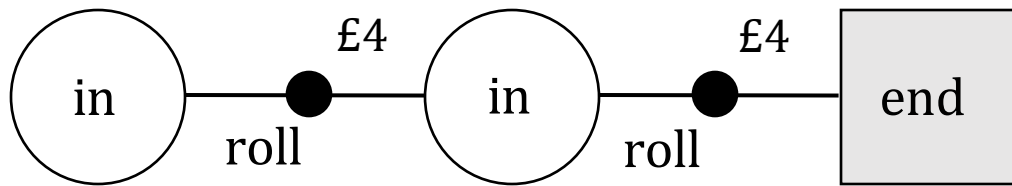




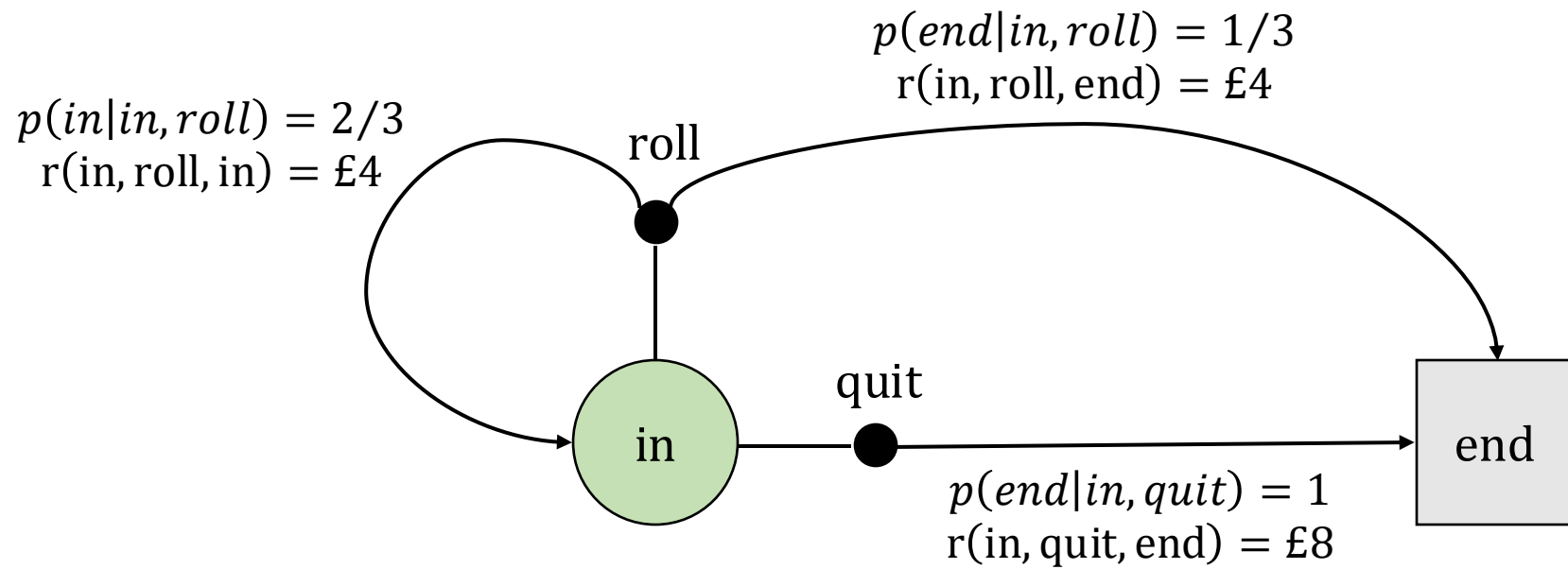
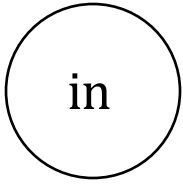


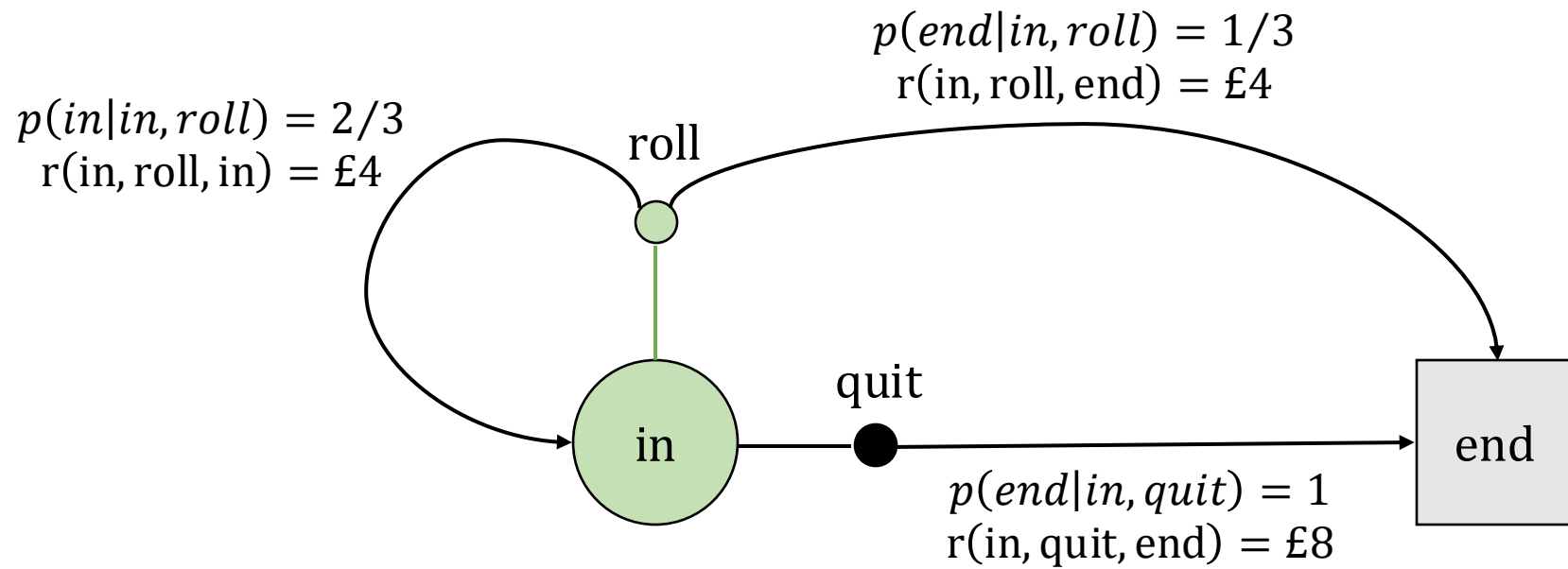
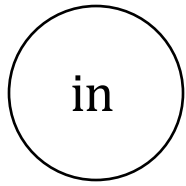


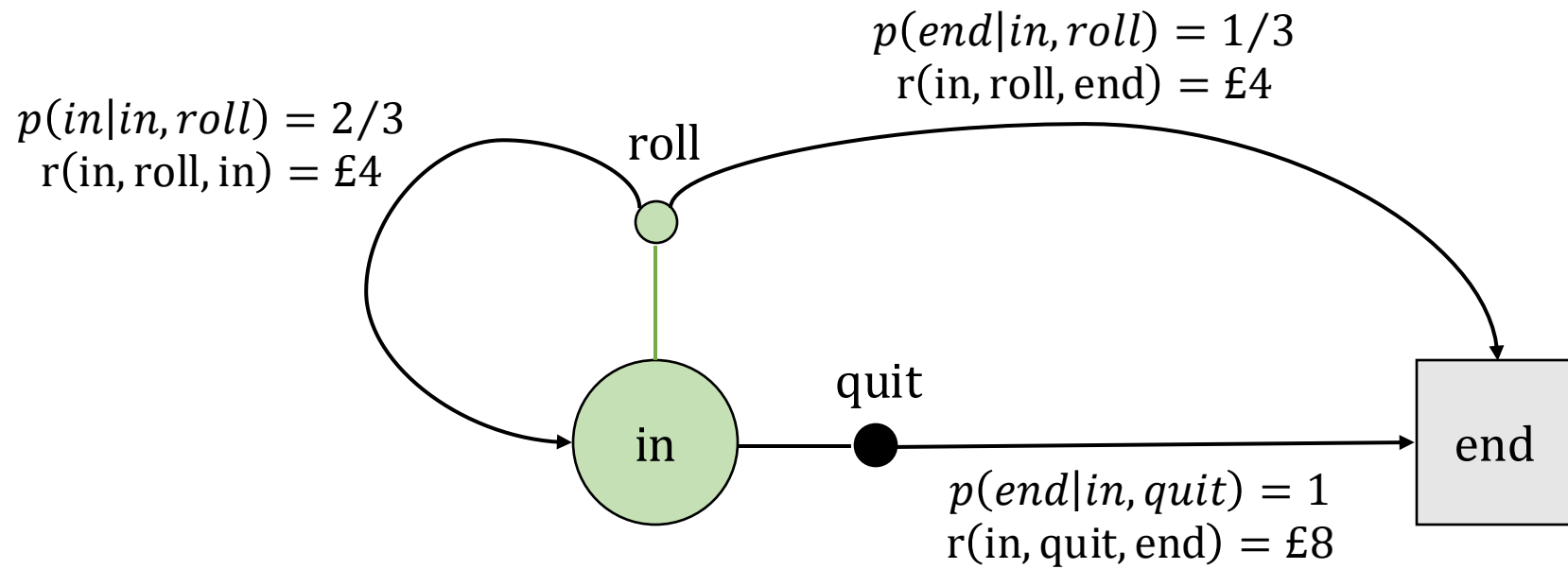
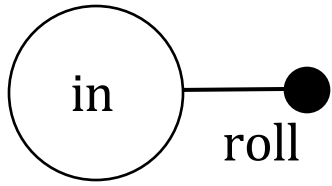


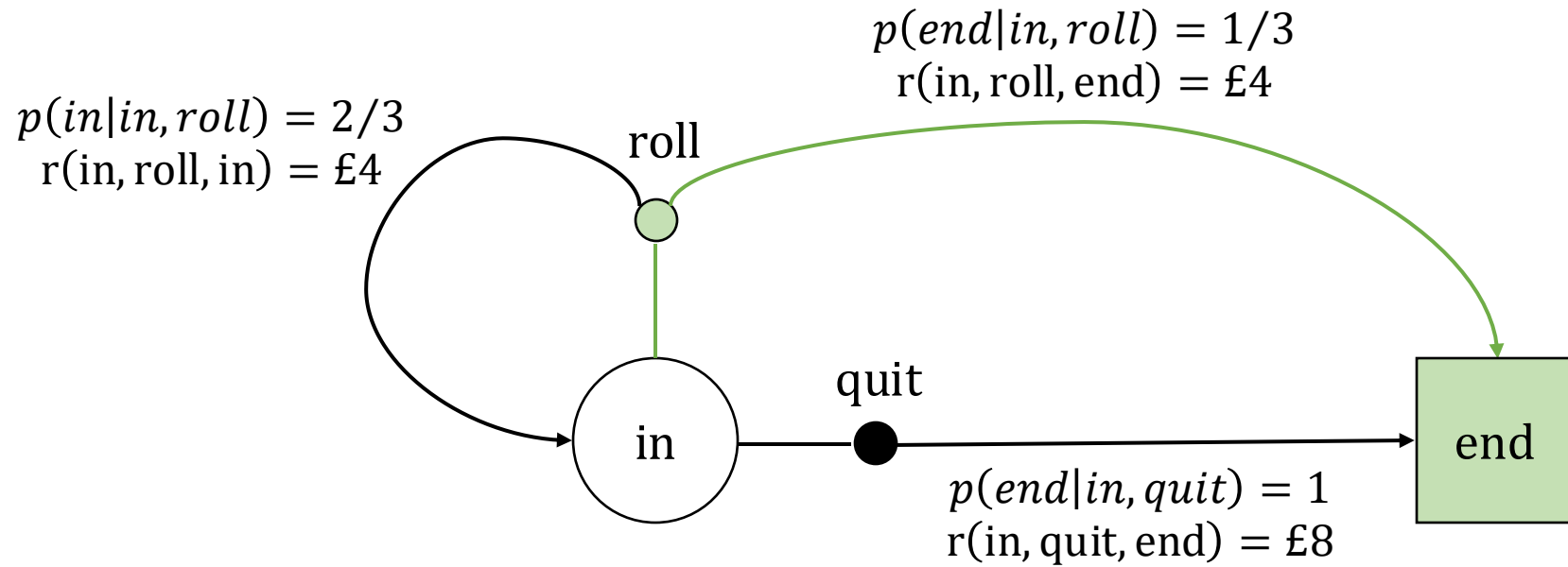
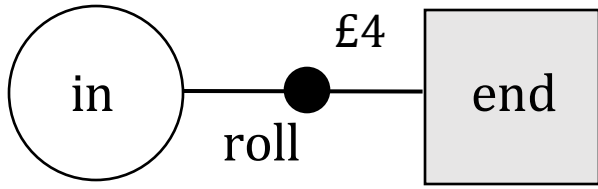


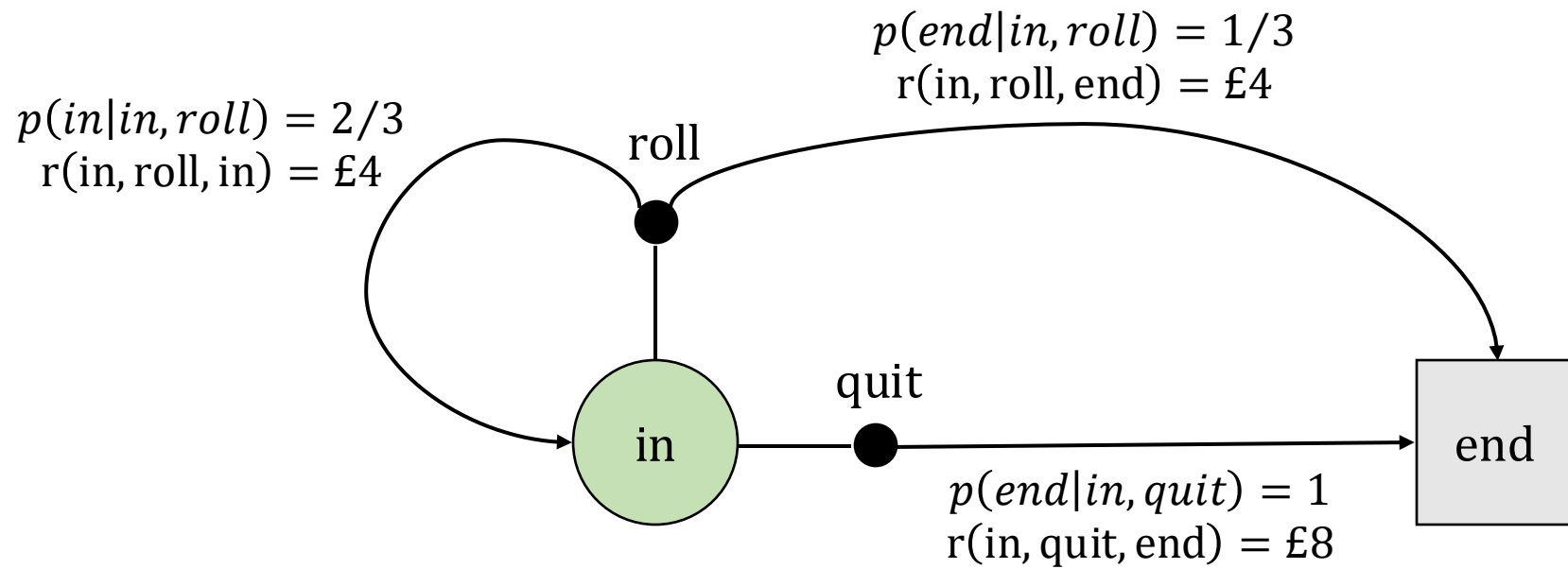
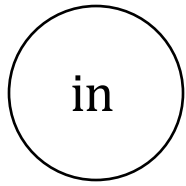


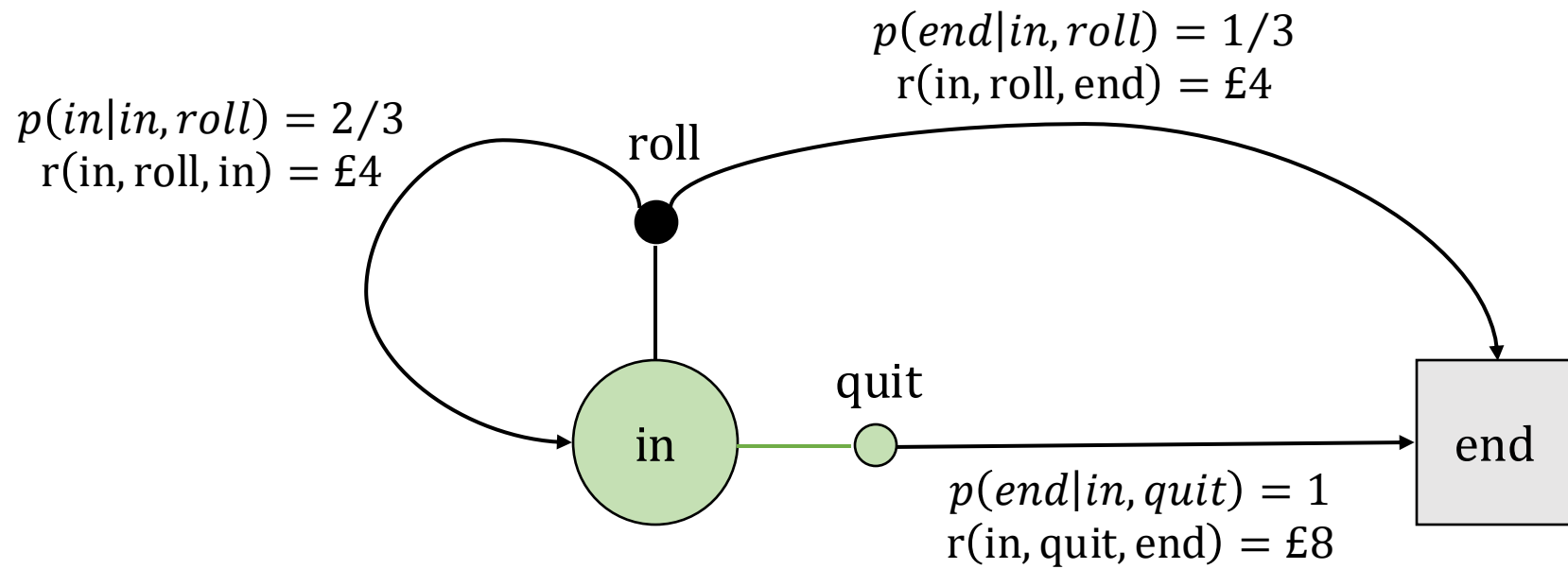
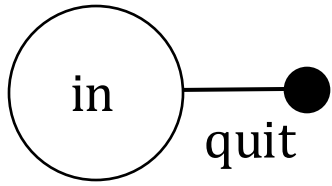


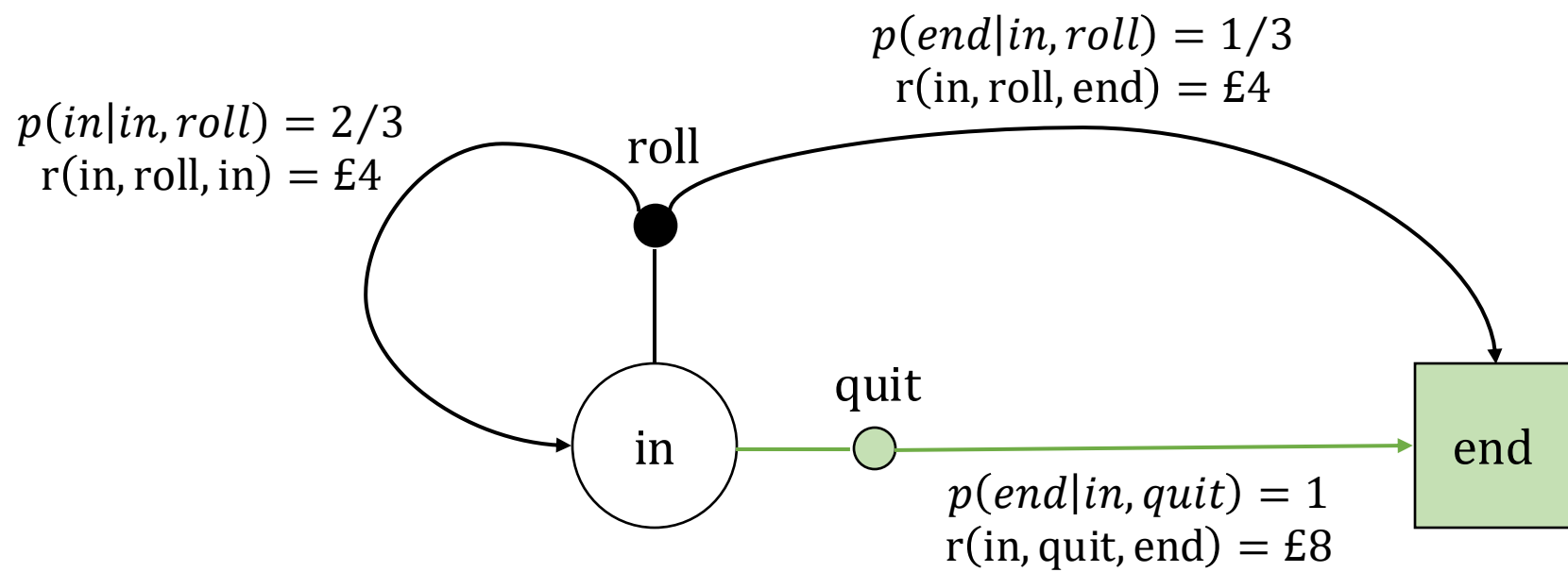
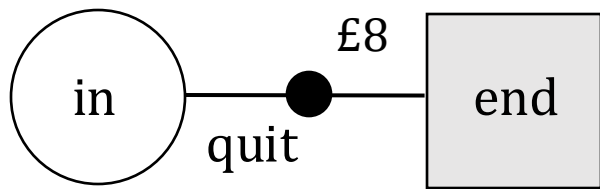


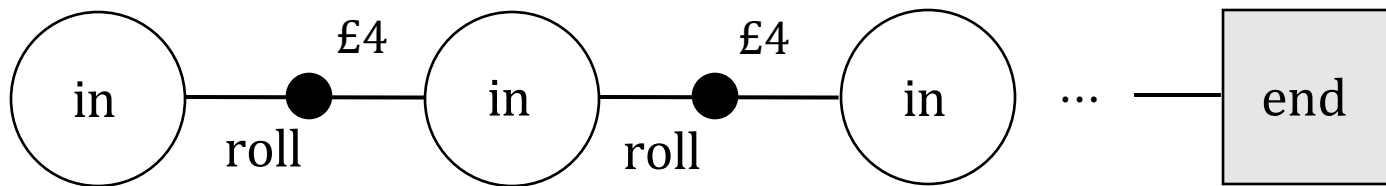




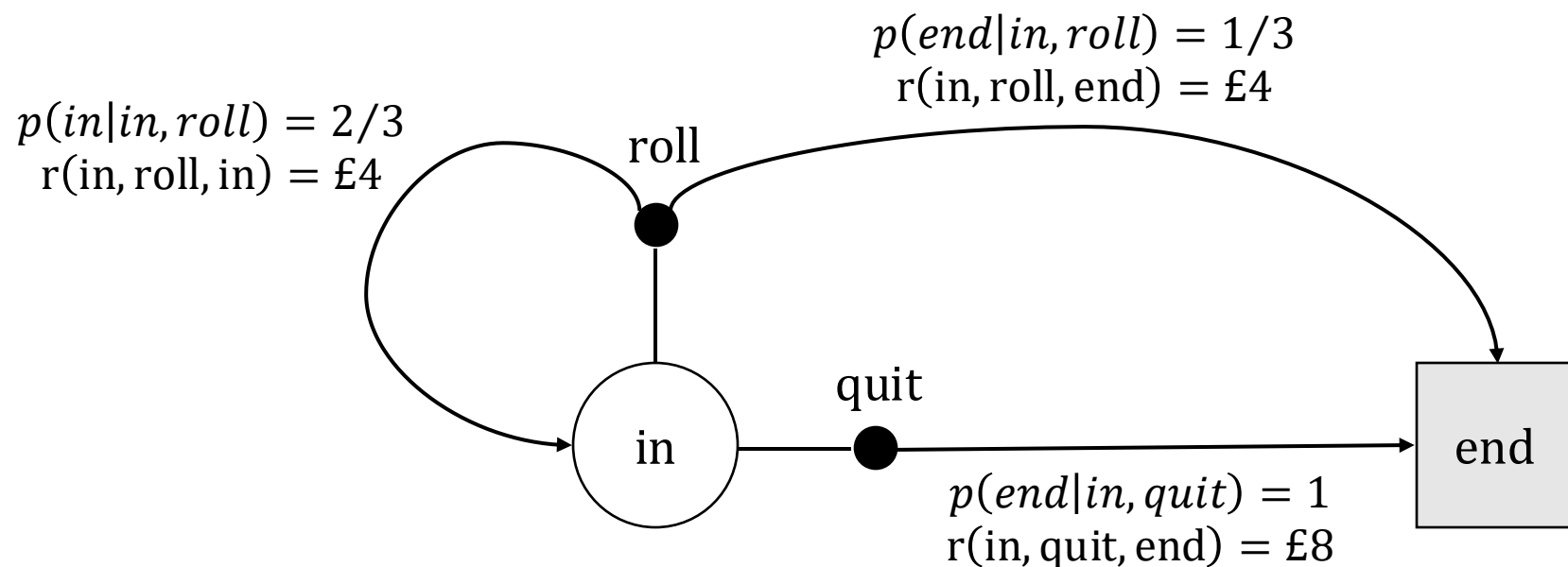








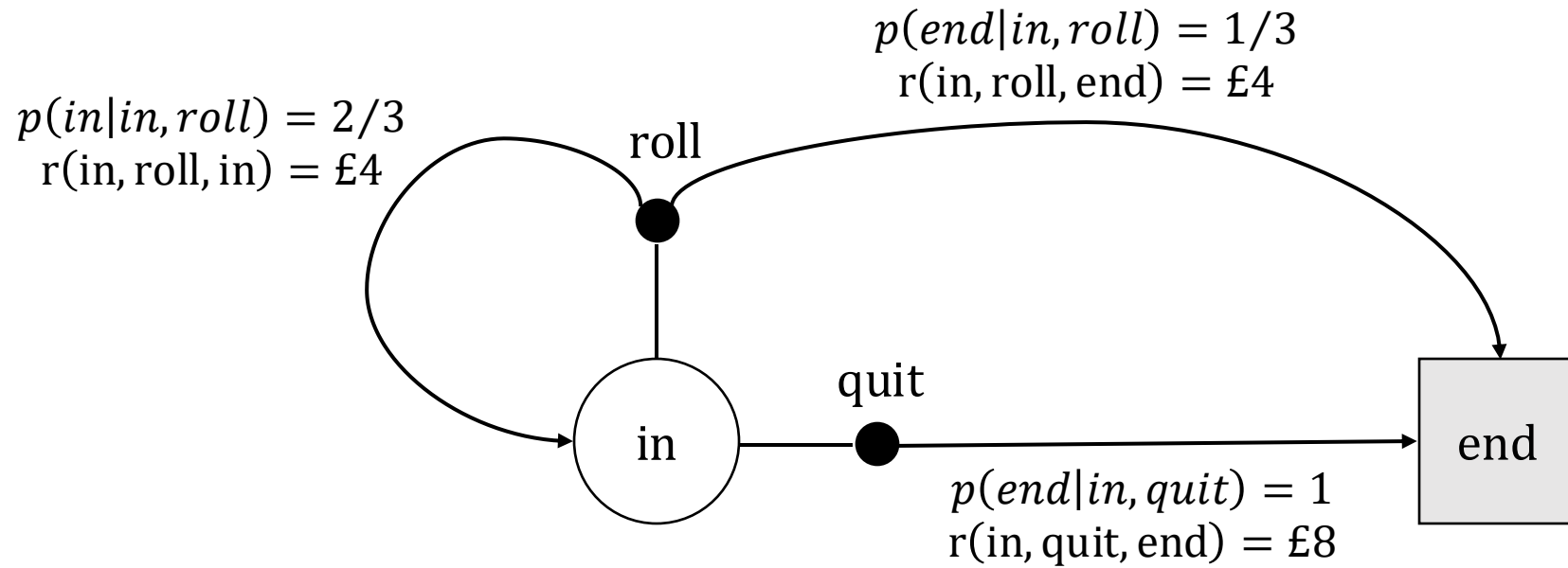
“Realisation”

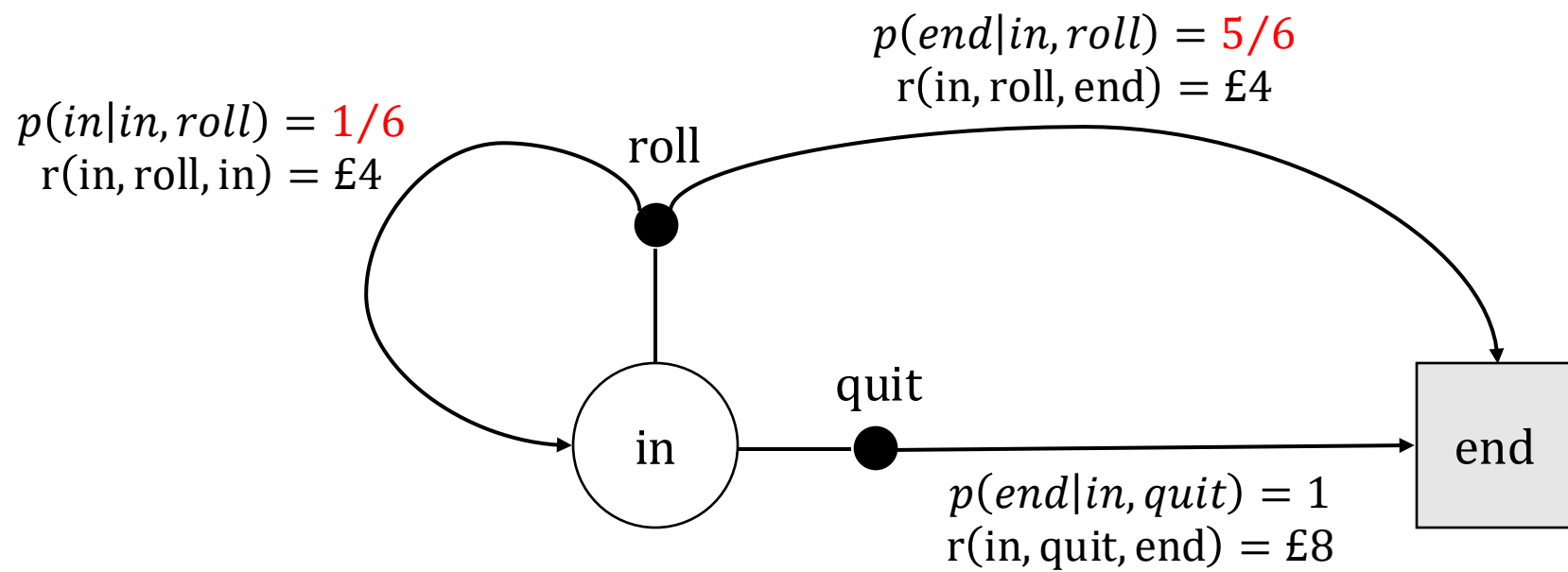


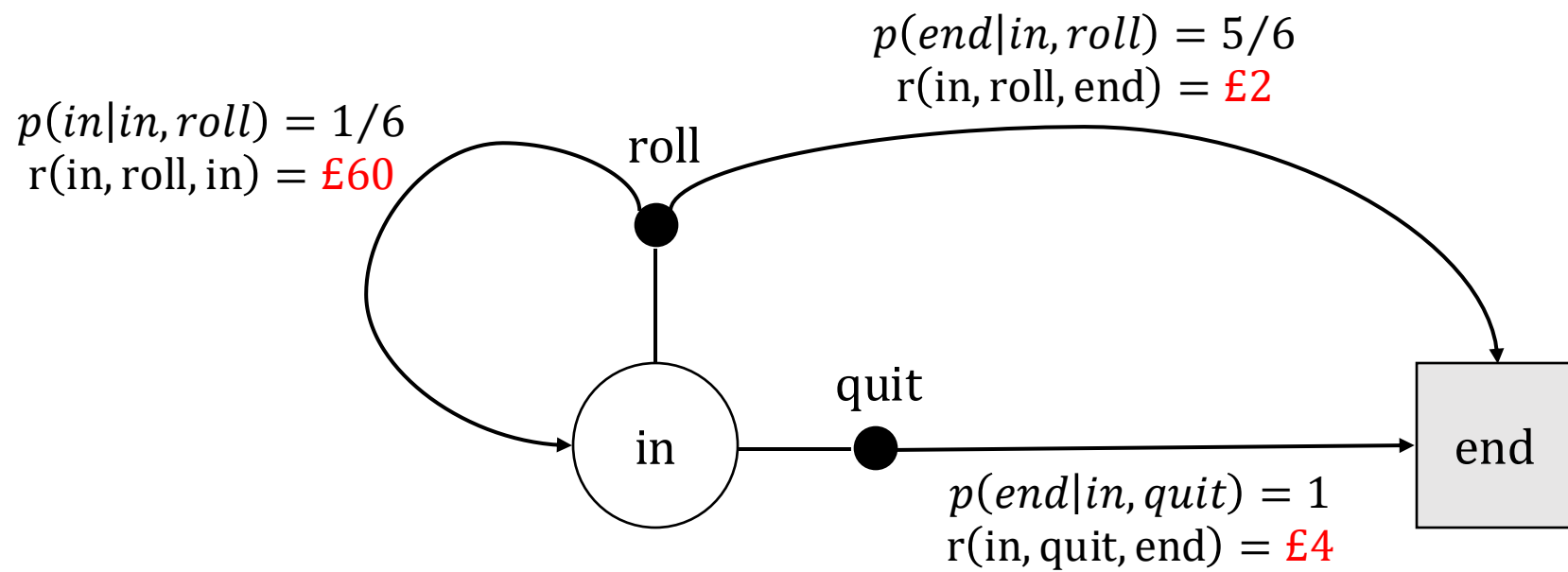
“Underlying phenomena”



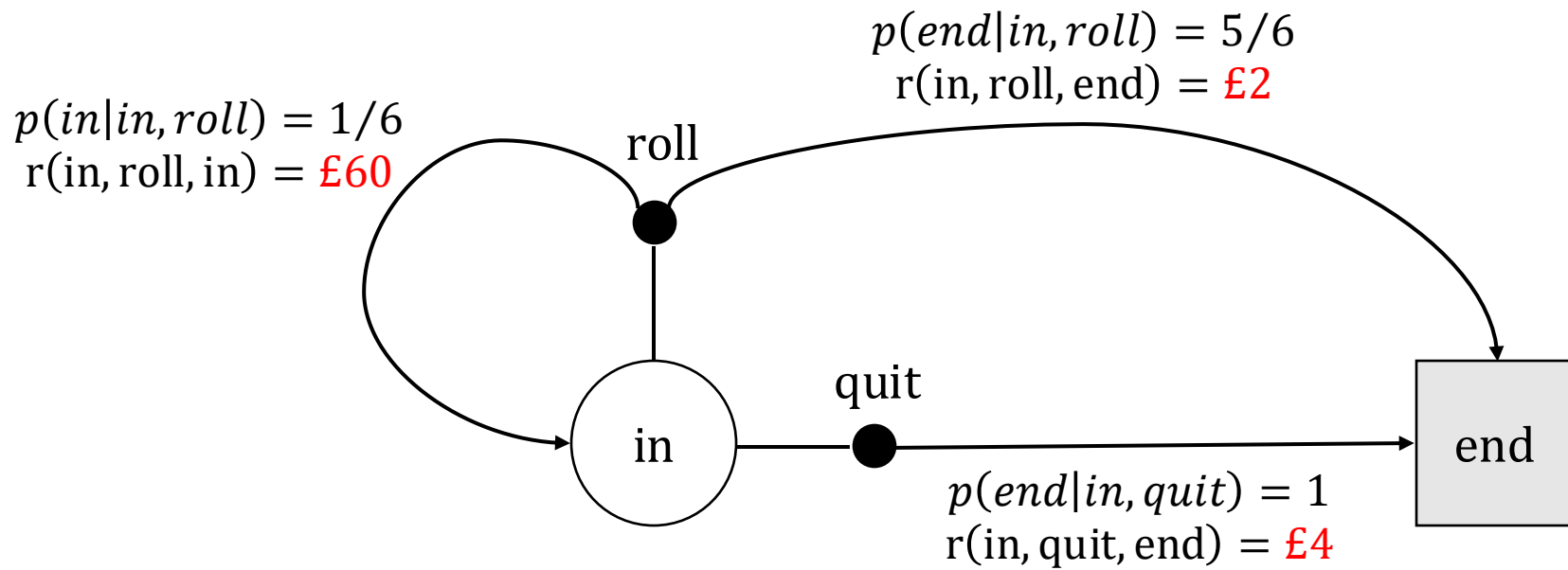
# How would you act?





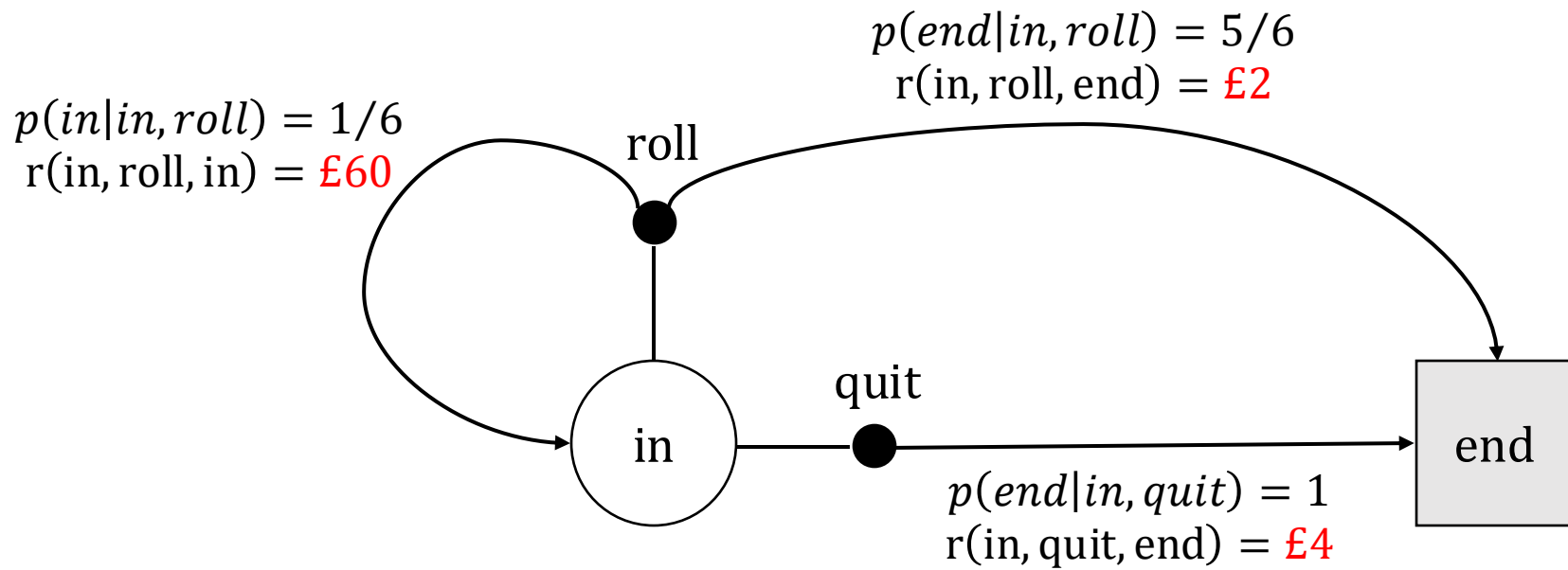


Our decision making will depend on both the transition probabilities and the rewards!



Our decision making will depend on both the transition probabilities and the rewards!

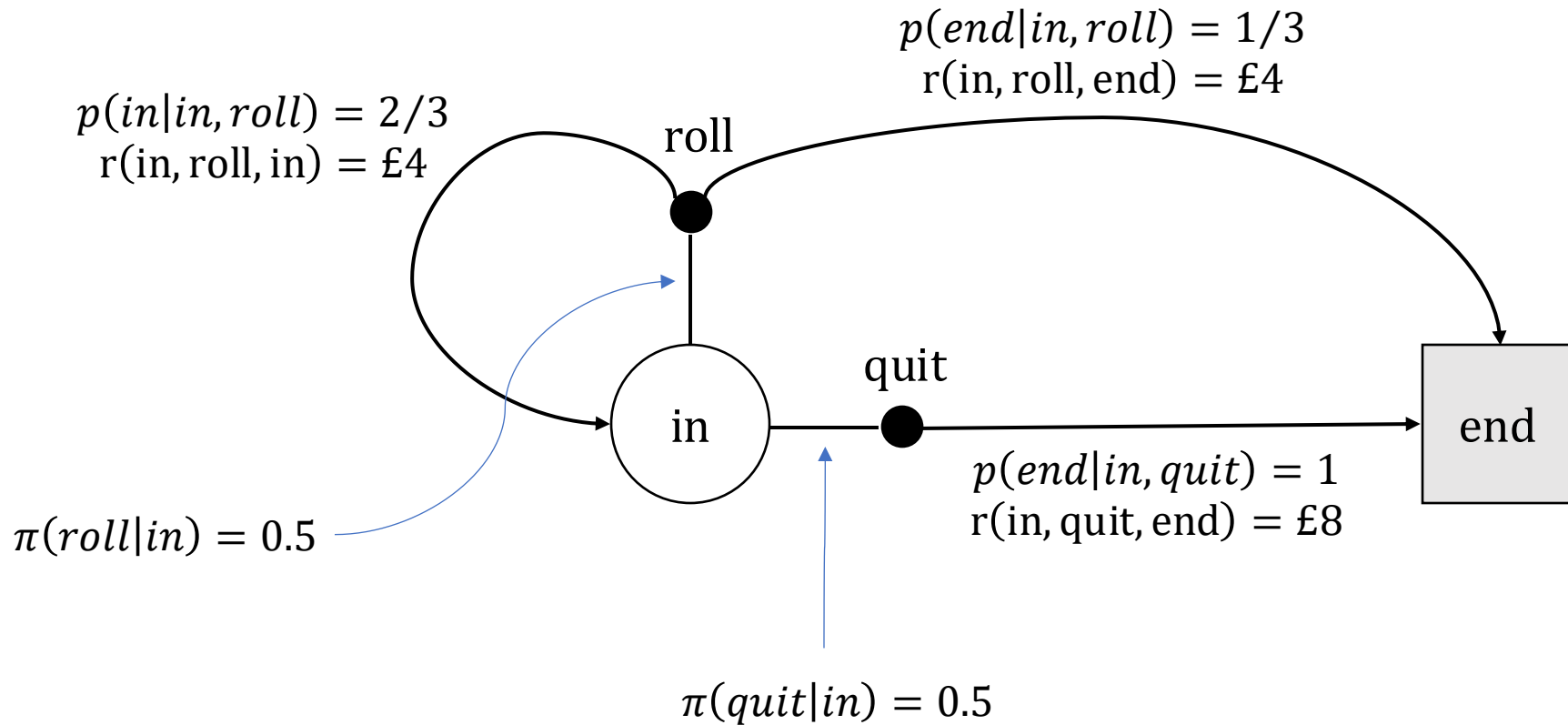
Optimal behaviour balances risks and rewards



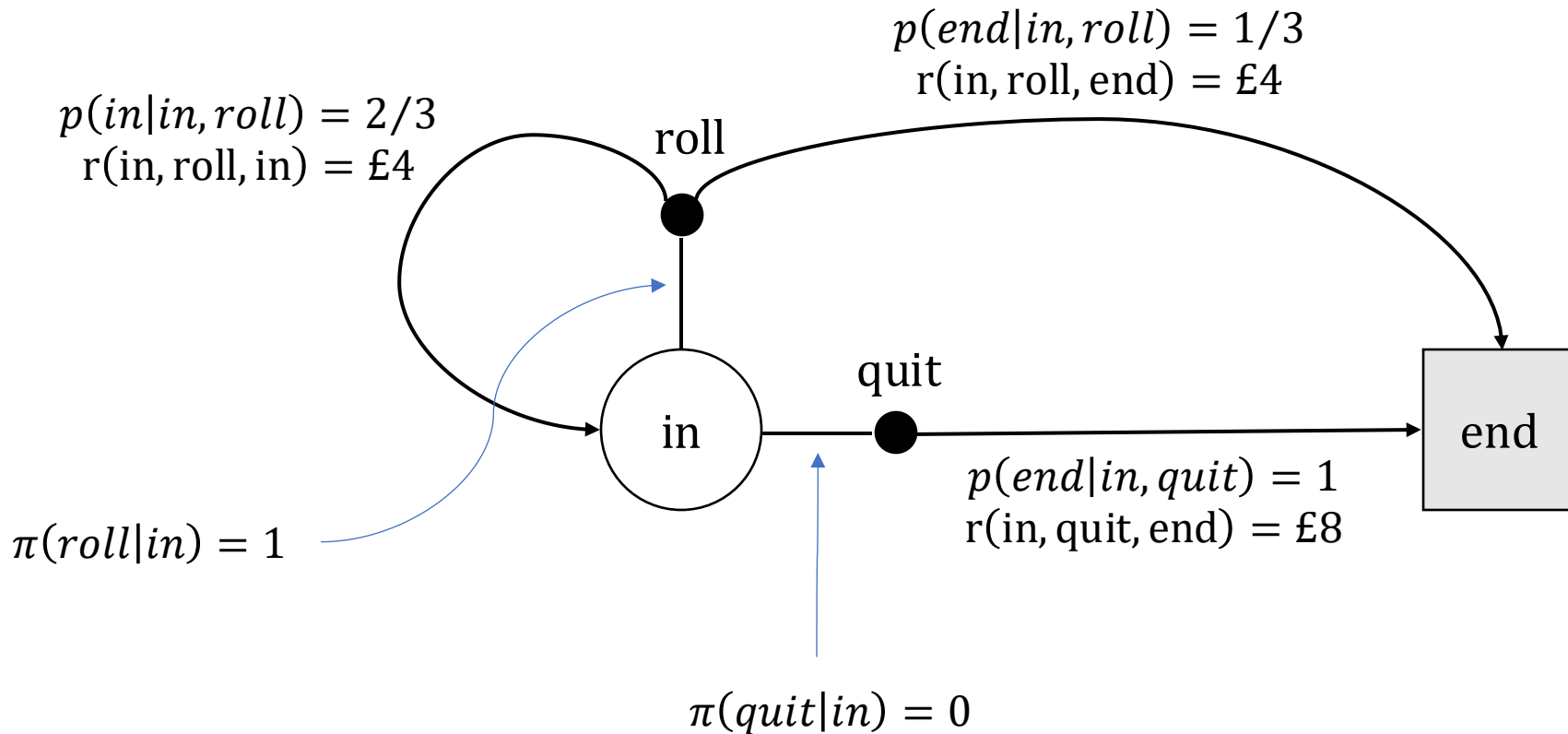
# Policies for MDPs

- The policy specifies which action to take, for each possible state  $\pi(a|s)$
- A **deterministic policy**  $\pi$  selects one possible action for each state.
- There can be policies which are not deterministic, called **stochastic policies**.
- A deterministic policy doesn't fully determine the path through an MDP, since actions still can have probabilistic outcomes.
- For example, in the simple rolling a die game, there are two possible deterministic policies: "Always Roll" and "Always Quit".

# Example - Completely random policy (stochastic)



# Example - “Always roll” policy (deterministic)





# Example policies for rolling a die game

Stochastic policy  $\pi_1$

| $\pi_1$   | $a = roll$ | $a = quit$ |
|-----------|------------|------------|
| $s = in$  | 0.5        | 0.5        |
| $s = end$ | 0          | 0          |

Deterministic policies  $\pi_2$  and  $\pi_3$

| $\pi_2$   | $a = roll$ | $a = quit$ |
|-----------|------------|------------|
| $s = in$  | 1          | 0          |
| $s = end$ | 0          | 0          |

| $\pi_3$   | $a = roll$ | $a = quit$ |
|-----------|------------|------------|
| $s = in$  | 0          | 1          |
| $s = end$ | 0          | 0          |

# Example policies for rolling a die game

Stochastic policy  $\pi_1$

| $\pi_1$   | $a = roll$ | $a = quit$ |
|-----------|------------|------------|
| $s = in$  | 0.5        | 0.5        |
| $s = end$ | 0          | 0          |

- We wish to list the probabilities for all actions, for every possible state.
- When we have chosen a policy, we know exactly how to act in each state

Deterministic policies  $\pi_2$  and  $\pi_3$

| $\pi_2$   | $a = roll$ | $a = quit$ |
|-----------|------------|------------|
| $s = in$  | 1          | 0          |
| $s = end$ | 0          | 0          |

| $\pi_3$   | $a = roll$ | $a = quit$ |
|-----------|------------|------------|
| $s = in$  | 0          | 1          |
| $s = end$ | 0          | 0          |

# Example policies for rolling a die game

Stochastic policy  $\pi_1$

| $\pi_1$          | $a = \text{roll}$ | $a = \text{quit}$ |
|------------------|-------------------|-------------------|
| $s = \text{in}$  | 0.5               | 0.5               |
| $s = \text{end}$ | 0                 | 0                 |

- We wish to list the probabilities for all actions, for every possible state.
- When we have chosen a policy, we know exactly how to act in each state

Deterministic policies  $\pi_2$  and  $\pi_3$

| $\pi_2$          | $a = \text{roll}$ | $a = \text{quit}$ |
|------------------|-------------------|-------------------|
| $s = \text{in}$  | 1                 | 0                 |
| $s = \text{end}$ | 0                 | 0                 |

| $\pi_3$          | $a = \text{roll}$ | $a = \text{quit}$ |
|------------------|-------------------|-------------------|
| $s = \text{in}$  | 0                 | 1                 |
| $s = \text{end}$ | 0                 | 0                 |

Next week we will move on to see how we can specify the best possible policy (i.e. action making) by taking the uncertainty into account in a structured way.

# Dicegame Jupyter Notebook

- In the notebook, released last week, you are presented with a slightly more complicated MDP than we have seen.
- Three dice are rolled, and the possible actions are rerolling any subset of the dice, including rerolling none and ending the game.
  - If you choose to reroll any dice, receive a penalty of -1.
  - If you choose not to reroll, turn any repeated rolls upside down, add up the total shown, and receive this reward.  
E.g. 2,2,4 is changed to 5,5,4 for a reward of 14.
  - The penalty/reward is added to your score.
- Next week, we will see how to obtain the best possible strategy for this game, using value iteration. But the first two tasks in the notebook should be possible before then – trying to find a good strategy by reasoning and trial and error.

Questions?

